

```

/**
 *
=====
=====
=====
* || NERKANAL – HRMS + ATS + Candidate
Portal (NestJS, single-file edition) ||
* || 15 modules · Jarvis AI assistant ·
Approvals engine · WebRTC meet · Cron ||
*
=====
=====
=====
* Run: npm i && npm run start →
http://localhost:3000 (Swagger: /docs)
* See NERKANAL_README.md for setup, env
vars, module map and deployment.
*
* MODULE MAP (search for the banner "$n"):
* $0 Core (entities, helpers, guards,
notifier, approvals engine, meet gateway)
* $1 Candidate Portal $2 Recruiter
ATS $3 Onboarding $4 Attendance
* $5 Leave $6
Payroll $7 Tracking+Claims $8
Performance
* $9 Engagement+Feed $10
Training $11 Complaints $12
Exit
* $13 HR Handbook $14

```

```

Reports          §15 Settings+Plans
(+Jarvis, Meet, Cron, Seed)
*/
import 'dotenv/config';
import 'reflect-metadata';
import * as fs from 'fs';
import * as path from 'path';
import * as crypto from 'crypto';
import * as bcrypt from 'bcryptjs';
import * as nodemailer from 'nodemailer';
import { diskStorage } from 'multer';
import { Server, Socket } from 'socket.io';
import {
  Module, Global, Injectable, Controller,
  Get, Post, Put, Delete, Body, Param, Query,
  Res, UseGuards,
  CanActivate, ExecutionContext,
  SetMetadata, ForbiddenException,
  BadRequestException, NotFoundException,
  UnauthorizedException, ConflictException,
  ValidationPipe, Logger, UseInterceptors,
  UploadedFile,
  OnModuleInit, createParamDecorator,
  applyDecorators,
} from '@nestjs/common';
import { NestFactory, Reflector, ModuleRef,
  APP_GUARD } from '@nestjs/core';
import { NestExpressApplication } from
  '@nestjs/platform-express';
import { FileInterceptor } from
  '@nestjs/platform-express';
import { TypeOrmModule, InjectRepository }
  from '@nestjs/typeorm';
import {

```

```

    Entity, PrimaryGeneratedColumn, Column,
    Repository, In, Like, Index, BeforeInsert,
  } from 'typeorm';
import { JwtModule, JwtService } from
 '@nestjs/jwt';
import { ScheduleModule, Cron } from
 '@nestjs/schedule';
import { SwaggerModule, DocumentBuilder,
 ApiTags, ApiBearerAuth } from
 '@nestjs/swagger';
import { WebSocketGateway, WebSocketServer,
 SubscribeMessage, OnGatewayConnection,
 OnGatewayDisconnect } from
 '@nestjs/websockets';

process.env.TZ = process.env.TZ ||
 'Asia/Kolkata';
const log = new Logger('Nerkanal');

/* _____ $0
HELPERS _____
*/
const ENV = (k: string, d = '') =>
 process.env[k] ?? d;
const BASE_URL = () => ENV('BASE_URL',
 'http://localhost:3000');
const pad = (n: number) =>
 String(n).padStart(2, '0');
const P = (d: any) => (typeof d ===
 'string' && d.length === 10 ? new Date(d +
 'T12:00:00') : new Date(d));
const ymd = (d: any = new Date()) => {
  const x = P(d); return
  `${x.getFullYear()}-${pad(x.getMonth() +

```

```

1)}}-${pad(x.getDate())}`; };
const now = () => new Date().toISOString();
const addDays = (d: any, n: number) => {
  const x = P(d); x.setDate(x.getDate() + n);
  return ymd(x); };
const addMonths = (d: any, n: number) => {
  const x = P(d); x.setMonth(x.getMonth() +
n); return x.toISOString(); };
const diffDays = (a: any, b: any) =>
Math.round((+P(b) - +P(a)) / 864e5);
const ym = (d: any = new Date()) =>
ymd(d).slice(0, 7);
const daysInMonth = (m: string) => new
Date(+m.slice(0, 4), +m.slice(5, 7),
0).getDate();
const round2 = (n: number) => Math.round((n
+ Number.EPSILON) * 100) / 100;
const rid = (n = 10) =>
crypto.randomBytes(n).toString('base64url')
.slice(0, n);
const sha = (s: string) =>
crypto.createHash('sha256').update(s).dige
st('hex');
const hav = (a: number, b: number, c:
number, d: number) => {
  const R = 6371000, t = (x: number) => (x
* Math.PI) / 180, dl = t(c - a), dg = t(d -
b);
  const h = Math.sin(dl / 2) ** 2 +
Math.cos(t(a)) * Math.cos(t(c)) *
Math.sin(dg / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
};
const mapUrl = (lat: number, lng: number)

```

```

=> `https://www.google.com/maps?
q=${lat},${lng}`;
const mapEmbed = (lat: number, lng: number)
=> `https://maps.google.com/maps?
q=${lat},${lng}&z=16&output=embed`;
const csv = (rows: any[][]) => rows.map(r
=> r.map(c => ` ${String(c ??
'')).replace(/"/g,
'""')})"`.join(',')).join('\n');
const ROLES = ['SUPER_ADMIN', 'ADMIN',
'HR_MANAGER', 'HRBP', 'HR', 'RM',
'DEPT_HEAD', 'RECRUITER', 'PANEL',
'FINANCE', 'EMPLOYEE', 'CANDIDATE'];
const HRS = ['HR', 'HRBP', 'HR_MANAGER'];
const ADMINS = ['ADMIN', 'SUPER_ADMIN'];
const MODULE_KEYS = ['ats', 'onboarding',
'attendance', 'leave', 'payroll',
'tracking', 'performance', 'engagement',
'training', 'complaints', 'exit',
'handbook', 'reports'];
const SIGNS = ['Aquarius', 'Pisces',
'Aries', 'Taurus', 'Gemini', 'Cancer',
'Leo', 'Virgo', 'Libra', 'Scorpio',
'Sagittarius', 'Capricorn']; // sign that
STARTS in month 1..12
const SIGN_CUT = [20, 19, 21, 20, 21, 21,
23, 23, 23, 23, 22, 22];
const zodiac = (dob: string) => {
  if (!dob) return null; const d = P(dob),
m = d.getMonth(), day = d.getDate();
  return day >= SIGN_CUT[m] ? SIGNS[m] :
SIGNS[(m + 11) % 12];
};
const horoscope = (sign: string, date =

```

```

ymd()) => {
  const h = parseInt(sha(sign +
date).slice(0, 8), 16);
  const pick = (a: string[], k: number) =>
a[(h >> k) % a.length];
  const colors = ['Navy', 'Gold',
'Emerald', 'Crimson', 'Ivory', 'Teal',
'Violet'];
  return {
    sign, date, mood: pick(['Energetic',
'Calm', 'Focused', 'Curious', 'Confident'],
1),
    text: `${pick(['A steady day for deep
work.', 'Collaboration brings a pleasant
surprise.', 'A small win today builds
momentum.', 'Listen first – the best idea
may come from a colleague.', 'Plan ahead
and the afternoon flows easily.'], 3)}
${pick(['Keep hydrated.', 'Take a short
walk after lunch.', 'Express gratitude to a
teammate.', 'Avoid multitasking.'], 5)}`,
    luckyNumber: (h % 9) + 1, luckyColor:
pick(colors, 7), note: 'For entertainment
only.',
  };
};
const aesKey = () =>
crypto.createHash('sha256').update(ENV('JWT
_SECRET', 'nerkanal-dev-secret')).digest();
const enc = (s: string) => { const iv =
crypto.randomBytes(12), c =
crypto.createCipheriv('aes-256-gcm',
aesKey(), iv); const e =
Buffer.concat([c.update(s), c.final()]);

```

```

return Buffer.concat([iv, c.getAuthTag(),
e]).toString('base64'); });
const dec = (s: string) => { const b =
Buffer.from(s, 'base64'), d =
crypto.createDecipheriv('aes-256-gcm',
aesKey(), b.subarray(0, 12));
d.setAuthTag(b.subarray(12, 28)); return
Buffer.concat([d.update(b.subarray(28)),
d.final()]).toString(); };

/** Safe arithmetic evaluator for salary
formulas: + - * / ( ) numbers, variables,
MIN/MAX/ROUND. */
function evalExpr(expr: string, vars:
Record<string, number>): number {
  const FN: any = { MIN: 'Math.min', MAX:
'Math.max', ROUND: 'Math.round' };
  const code = String(expr).replace(/[A-Za-
z_][A-Za-z0-9_]*\b/g, (m) => {
    if (FN[m.toUpperCase()]) return
FN[m.toUpperCase()];
    if (!(m in vars)) throw new
BadRequestException(`Unknown variable
"${m}" in formula "${expr}"`);
    return
`${Number(vars[m]).toFixed(4)}`;
  });
  if (!/^[0-9+\-
*/(.\, \s]*$/.test(code.replace(/Math\.(
min|max|round)/g, ''))) throw new
BadRequestException('Invalid formula');
  const v = Function(`"use strict";return
(${code})` )();
  if (!Number.isFinite(v)) throw new

```

```

BadRequestException(`Formula "${expr}" did
not evaluate to a number`);
    return v;
}

/* _____ $0
ENTITIES _____
*/
const S = (o: any = {}) => Column({ type:
'varchar', nullable: true, ...o });
const J = () => Column({ type: 'simple-
json', nullable: true });
const N = (d = 0) => Column({ type: 'int',
default: d });
const F = (d = 0) => Column({ type:
'float', default: d });
const B = (d = false) => Column({ type:
'boolean', default: d });
class Base {
    @PrimaryGeneratedColumn('uuid') id:
string;
    @Column({ type: 'varchar' }) createdAt:
string;
    @BeforeInsert() ts() { this.createdAt =
this.createdAt || now(); }
}
@Entity() class Company extends Base {
    @S() name: string; @S() state: string;
    @S() country: string; @S() deviceKey:
string; @S({ default: 'FREE' }) atsPlan:
string;
    @J() modules: string[]; @J() settings:
any;
}

```

```

@Entity() class User extends Base {
    @Index() @S() companyId: string;
    @Column({ type: 'varchar', unique: true })
    email: string; @S() passwordHash: string;
    @S() name: string; @S({ default:
    'CANDIDATE' }) role: string; @S({ default:
    'CANDIDATE' }) kind: string; @S() phone:
    string;
    @S() dob: string; @S() zodiac: string;
    @S() otp: string; @S() otpExp: string; @S()
    empId: string; @S() rmId: string; @S()
    hrbpId: string;
    @S() designation: string; @S()
    department: string; @S() doj: string; @S()
    probationEnd: string; @S() probationState:
    string;
    @S({ default: 'NA' }) onboarding: string;
    @S() videoLockUntil: string; @S()
    candidateId: string;
    @B() verified: boolean; @B(true) active:
    boolean; @B() mustChangePwd: boolean; @N()
    profilePct: number; @N() videoAttempts:
    number;
    @J() profile: any; @J() docs: any[]; @J()
    video: any;
}
@Entity() class Job extends Base {
    @Index() @S() companyId: string; @S()
    title: string; @S() location: string; @S({
    type: 'text' }) description: string;
    @S({ default: 'INTERNAL' }) source:
    string; @S() applyUrl: string; @S() reqId:
    string; @S() recruiterId: string;
    @S({ default: 'OPEN' }) status: string;

```

```

@S() externalId: string; @S() companyName:
string; @J() skills: string[];
}
@Entity() class Application extends Base {
  @Index() @S() companyId: string; @S()
  jobId: string; @S() candidateId: string;
  @S({ default: 'APPLIED' }) status: string;
  @J() timeline: any[]; @J() docs: any[];
  @S({ type: 'text' }) notes: string;
}
@Entity() class Meeting extends Base {
  @Index() @S() companyId: string; @S()
  kind: string; @S() title: string; @S()
  hostId: string; @S() roomId: string; @S()
  link: string;
  @S() startsAt: string; @N(30)
  durationMin: number; @S({ default:
  'SCHEDULED' }) status: string; @J()
  attendeeIds: string[]; @J() ctx: any;
}
@Entity() class Approval extends Base {
  @Index() @S() companyId: string; @S()
  type: string; @S() requesterId: string;
  @S() title: string;
  @S({ default: 'PENDING' }) status:
  string; @N() step: number; @J() chain:
  string[]; @J() payload: any; @J() history:
  any[];
}
@Entity() class Attendance extends Base {
  @Index() @S() companyId: string; @Index()
  @S() userId: string; @S() date: string;
  @S() status: string; @S() mode: string;
  @S() inAt: string; @S() outAt: string;

```

```

@S() mood: string; @F() lat: number; @F()
lng: number; @J() flags: any;
}
@Entity() class LeaveType extends Base {
  @Index() @S() companyId: string; @S()
code: string; @S() name: string; @F()
quota: number; @B() carry: boolean;
  @B(true) paid: boolean; @J()
applicableTo: string[]; @N(0)
proofAfterDays: number;
}
@Entity() class LeaveBal extends Base {
@Index() @S() userId: string; @S() code:
string; @S() year: string; @F() bal:
number; }
@Entity() class Notification extends Base {
  @Index() @S() userId: string; @S() title:
string; @S({ type: 'text' }) body: string;
@S() type: string; @S() link: string; @S()
dedupe: string; @B() read: boolean;
}
@Entity() class Rec extends Base { //
generic record store: KPI, PIP, COMPLAINT,
POLICY, ACK, STRUCT, PAYRUN, TRIP, ...
  @Index() @S() companyId: string; @Index()
@S() type: string; @S() ownerId: string;
@S() refId: string; @S() key: string; @S()
status: string; @J() data: any;
}
const ENTITIES = [Company, User, Job,
Application, Meeting, Approval, Attendance,
LeaveType, LeaveBal, Notification, Rec];

/* _____ $0 CORE

```

SERVICES & GUARDS

```
_____*/  
@Injectable() class Db {  
    constructor(  
        @InjectRepository(Company) public  
companies: Repository<Company>,  
        @InjectRepository(User) public users:  
Repository<User>,  
        @InjectRepository(Job) public jobs:  
Repository<Job>,  
        @InjectRepository(Application) public  
apps: Repository<Application>,  
        @InjectRepository(Meeting) public  
meetings: Repository<Meeting>,  
        @InjectRepository(Approval) public  
approvals: Repository<Approval>,  
        @InjectRepository(Attendance) public  
att: Repository<Attendance>,  
        @InjectRepository(LeaveType) public  
leaveTypes: Repository<LeaveType>,  
        @InjectRepository(LeaveBal) public  
leaveBal: Repository<LeaveBal>,  
        @InjectRepository(Notification) public  
notes: Repository<Notification>,  
        @InjectRepository(Rec) public recs:  
Repository<Rec>,  
    ) {}  
    addRec(type: string, o: Partial<Rec>) {  
return this.recs.save(this.recs.create({  
type, data: {}, ...o } as any)) as  
Promise<Rec>; }  
    listRec(type: string, where: any = {}) {  
return this.recs.find({ where: { type,  
...where }, order: { createdAt: 'DESC' }  
}
```

```

}); }

  async rec(type: string, id: string) {
    const r = await this.recs.findOneBy({ id,
    type }); if (!r) throw new
    NotFoundException(`${type} not found`);
    return r; }

  async user(id: string) { const u = await
    this.users.findOneBy({ id }); if (!u) throw
    new NotFoundException('User not found');
    return u; }

  async company(id: string) { const c =
    await this.companies.findOneBy({ id }); if
    (!c) throw new NotFoundException('Company
    not found'); return c; }

  async byRole(companyId: string, roles:
    string[]) { return this.users.find({ where:
    { companyId, role: In(roles), active: true
    } }); }

  async ids(companyId: string, roles:
    string[]) { return (await
    this.byRole(companyId, roles)).map(u =>
    u.id); }

  async employees(companyId: string) {
    return this.users.find({ where: {
    companyId, kind: 'EMPLOYEE', active: true }
    }); }

  async teamOf(rmId: string) { return
    this.users.find({ where: { rmId, active:
    true } }); }
}

const clean = (u: User) => { if (!u) return
u; const { passwordHash, otp, otpExp, ...r
} = u as any; return r; };

```

```

/** Live socket.io server holder (set by
the gateway). */
const Live: { io?: Server } = {};
@Injectable() class Notifier {
  private tx =
nodemailer.createTransport(ENV('SMTP_URL')
|| ({ jsonTransport: true } as any));
  constructor(private db: Db) {}
  /** In-app + live push + email (+SMS if
Twilio configured). Use `dedupe` to make
cron triggers idempotent. */
  async send(to: string | string[], n: {
title: string; body?: string; type?:
string; link?: string; dedupe?: string;
email?: boolean; sms?: boolean }) {
    const ids = [...new
Set((Array.isArray(to) ? to :
[to]).filter(Boolean))];
    for (const userId of ids) {
      try {
        if (n.dedupe && (await
this.db.notes.findOneBy({ userId, dedupe:
n.dedupe }))) continue;
        const row = await
this.db.notes.save(this.db.notes.create({
userId, title: n.title, body: n.body || '',
type: n.type || 'INFO', link: n.link,
dedupe: n.dedupe, read: false }));

Live.io?.to(`user:${userId}`).emit('notify'
, row);

        const u = await
this.db.users.findOneBy({ id: userId });
        if (u && n.email !== false)

```

```

this.tx.sendMail({ from: ENV('MAIL_FROM',
'Nerkanal <no-reply@nerkanal.app>'), to:
u.email, subject: n.title, text: `${n.body
|| ''}\n${n.link ? BASE_URL() + n.link :
''}` })
    .then(i => !ENV('SMTP_URL') &&
log.debug(`MAIL → ${u.email}:
${n.title}`)).catch(e => log.warn('mail
failed ' + e.message));
    if (u?.phone && n.sms)
this.sms(u.phone, `${n.title} ${n.body ||
''}`.slice(0, 300));
    } catch (e) { log.warn('notify
failed: ' + e.message); }
    }
}
    async sms(to: string, body: string) {
        const sid = ENV('TWILIO_SID'), tok =
ENV('TWILIO_TOKEN'), from =
ENV('TWILIO_FROM');
        if (!sid) return log.debug(`SMS →
${to}: ${body}`);
        await
fetch(`https://api.twilio.com/2010-04-
01/Accounts/${sid}/Messages.json`, {
            method: 'POST', headers: {
Authorization: 'Basic ' +
Buffer.from(`${sid}:${tok}`).toString('base
64'), 'Content-Type': 'application/x-www-
form-urlencoded' },
            body: new URLSearchParams({ To: to,
From: from, Body: body }),
        }).catch(e => log.warn('sms failed ' +
e.message));
    }
}

```

```

    }
}

/* Auth decorators + global access guard
(JWT → roles → purchased-module gate) */
const Public = () => SetMetadata('public',
true);
const Roles = (...r: string[]) =>
SetMetadata('roles', r);
const Feat = (f: string) =>
SetMetadata('feature', f);
const Me = createParamDecorator((_ : any,
ctx: ExecutionContext) =>
ctx.switchToHttp().getRequest().user);
const Ctl = (p: string) =>
applyDecorators(Controller(p), ApiTags(p),
ApiBearerAuth());
@Injectable() class AccessGuard implements
CanActivate {
    constructor(private ref: Reflector,
private jwt: JwtService, private db: Db) {}
    async canActivate(ctx: ExecutionContext)
    {
        const get = (k: string) =>
this.ref.getAllAndOverride<any>(k,
[ctx.getHandler(), ctx.getClass()]);
        if (get('public')) return true;
        const req =
ctx.switchToHttp().getRequest();
        const tok = (req.headers.authorization
|| '').replace(/^Bearer /i, '') ||
req.query?.token;
        if (!tok) throw new
UnauthorizedException('Login required');
    }
}

```

```

        let payload: any; try { payload =
this.jwt.verify(tok); } catch { throw new
UnauthorizedException('Session expired'); }
        const u = await
this.db.users.findOneBy({ id: payload.sub
});
        if (!u || !u.active) throw new
UnauthorizedException('Account inactive');
        req.user = u;
        const roles: string[] = get('roles');
        if (roles?.length &&
!roles.includes(u.role) && u.role !==
'SUPER_ADMIN') throw new
ForbiddenException(`Requires role:
${roles.join(' / ')}');
        const feat: string = get('feature');
        if (feat && u.companyId) {
            const c = await
this.db.company(u.companyId);
            if (!(c.modules ||
[]).includes(feat)) throw new
ForbiddenException(`Module "${feat}" is not
purchased by your organisation. Ask your
admin to subscribe (Settings → Plans).`);
        }
        return true;
    }
}

/** Generic multi-step approval engine used
by leave, attendance requests,
requisitions, claims, rewards, resignation,
termination. */
@Injectable() class Approvals {

```

```

    constructor(private db: Db, private n:
Notifier) {}
    private expand(role: string) { return
role === 'HR' ? HRS : role === 'ADMIN' ?
ADMINS : [role]; }
    async approvers(a: Approval, step =
a.step): Promise<User[]> {
        const role = a.chain[step], req = await
this.db.users.findOneBy({ id: a.requesterId
});
        if (role === 'RM' && req?.rmId) return
[await this.db.users.findOneBy({ id:
req.rmId })].filter(Boolean);
        if (role === 'HRBP' && req?.hrbpId)
return [await this.db.users.findOneBy({ id:
req.hrbpId })].filter(Boolean);
        const roles = role === 'RM' ? [...HRS]
: this.expand(role); // no RM set → falls
back to HR
        return this.db.byRole(a.companyId,
roles);
    }
    async create(type: string, req: User,
payload: any, chain: string[], title:
string) {
        const a = await
this.db.approvals.save(this.db.approvals.cr
eate({ companyId: req.companyId, type,
requesterId: req.id, title, chain, payload,
step: 0, status: 'PENDING', history: []
}));
        await this.notifyStep(a); return a;
    }
    async notifyStep(a: Approval) {

```

```

        const ap = await this.approvers(a);
        await this.n.send(ap.map(x => x.id), {
            title: `Approval needed: ${a.title}`, body:
            `Step ${a.step + 1}/${a.chain.length}
            (${a.chain[a.step]})`, type: 'APPROVAL',
            link: `/approvals/${a.id}`, sms: a.type ===
            'LEAVE' });
    }

    async canAct(a: Approval, u: User) {
        return u.role === 'SUPER_ADMIN' || (await
        this.approvers(a)).some(x => x.id ===
        u.id); }

    async view(a: Approval) {
        const pendingWith = a.status ===
        'PENDING' ? a.chain[a.step] : null;
        return { ...a, pendingWith,
        pendingWithNames: pendingWith ? (await
        this.approvers(a)).map(x => x.name) : [] };
    }

    async act(id: string, actor: User,
    approve: boolean, remarks = '') {
        const a = await
        this.db.approvals.findOneBy({ id }); if
        (!a) throw new NotFoundException('Approval
        not found');

        if (a.status !== 'PENDING') throw new
        BadRequestException(`Already ${a.status}`);
        if (a.companyId !== actor.companyId &&
        actor.role !== 'SUPER_ADMIN') throw new
        ForbiddenException();

        if (!(await this.canAct(a, actor)))
        throw new ForbiddenException('This is not
        your approval step');

        a.history = [...(a.history || []), {

```

```

step: a.step, role: a.chain[a.step], by:
actor.id, byName: actor.name, decision:
approve ? 'APPROVED' : 'REJECTED', remarks,
at: now() }]);
    if (!approve) {
        a.status = 'REJECTED'; await
this.db.approvals.save(a);
        if (a.type === 'ATT_REQ') { const r =
await this.db.users.findOneBy({ id:
a.requesterId }); if (r) await
this.markAtt(r.id, r.companyId,
a.payload.date, { status: 'ABSENT', flags:
{ rejected: true } }); } await
this.n.send(a.requesterId, { title:
`Rejected: ${a.title}`, body: remarks,
type: 'APPROVAL', link:
`/approvals/${a.id}`, sms: true }); return
a; }
        if (a.step + 1 >= a.chain.length) {
            a.status = 'APPROVED'; await
this.db.approvals.save(a);
            try { await this.hooks(a); } catch
(e) { log.error('approval hook failed: ' +
e.stack); }
            await this.n.send(a.requesterId, {
title: `Approved: ${a.title}`, body:
remarks, type: 'APPROVAL', link:
`/approvals/${a.id}`, sms: true });
        } else {
            a.step += 1; await
this.db.approvals.save(a); await
this.notifyStep(a);
            await this.n.send(a.requesterId, {
title: `${a.title}: moved to

```

```

    ${a.chain[a.step]}`, body: 'Previous step
    approved', type: 'APPROVAL', link:
    `/approvals/${a.id}` });
  }
  return a;
}
  async markAtt(userId: string, companyId:
  string, date: string, patch:
  Partial<Attendance>) {
    let r = await this.db.att.findOneBy({
  userId, date });
    if (!r) r = this.db.att.create({
  userId, companyId, date, flags: {} });
    Object.assign(r, patch); return
  this.db.att.save(r);
  }
  async hooks(a: Approval) {
    const p = a.payload || {}, u = await
  this.db.user(a.requesterId);
    switch (a.type) {
      case 'LEAVE': {
        const year =
  String(P(p.from).getFullYear());
        const b = await
  this.db.leaveBal.findOneBy({ userId: u.id,
  code: p.code, year });
        if (b) { b.bal = round2(b.bal -
  p.days); await this.db.leaveBal.save(b); }
        for (let d = p.from; d <= p.to; d =
  addDays(d, 1)) await this.markAtt(u.id,
  u.companyId, d, { status: p.paid === false
  ? 'LOP' : 'LEAVE', mode: 'LEAVE' });
        break;
      }
    }
  }
}

```

```

        case 'COMPOFF': {
            const year = String(new
Date().getFullYear());
            let b = await
this.db.leaveBal.findOneBy({ userId: u.id,
code: 'CO', year });
            if (!b) b =
this.db.leaveBal.create({ userId: u.id,
code: 'CO', year, bal: 0 });
            b.bal = round2(b.bal + (p.days ||
1)); await this.db.leaveBal.save(b); break;
        }
        case 'ATT_REQ': await
this.markAtt(u.id, u.companyId, p.date, {
status: p.toStatus || 'PRESENT', flags: {
approved: true, kind: p.kind }, ...(p.inAt
? { inAt: p.inAt, outAt: p.outAt } : {}))
}); break;
        case 'RESIGNATION': case
'TERMINATION': {
            const hrbp = u.hrbpId || (await
this.db.ids(u.companyId, HRS))[0];
            const ex = await
this.db.addRec('EXIT', { companyId:
u.companyId, ownerId: u.id, status:
'NOTICE', data: { kind: a.type, reason:
p.reason, requestedLwd: p.lwd, clearances:
{ KT: null, ASSET: null, ACCESS: null },
letters: {} } });
            const when = new Date(Date.now() +
864e5); when.setHours(15, 0, 0, 0);
            const m = await
this.meets(u.companyId, hrbp,
'EXIT_INTERVIEW', `Exit interview –

```

```

    ${u.name}`, when.toISOString(), 30, [u.id,
    hrbp], { exitId: ex.id });
        await this.n.send(await
    this.db.ids(u.companyId, HRS), { title:
    `Exit initiated: ${u.name}`, body: 'Set
    notice period / last working day', type:
    'EXIT', link: `/exit/${ex.id}` });
        break;
    }
    case 'REWARD': await
    this.n.send(a.requesterId, { title: 'Reward
    released to payroll/finance', type:
    'REWARD' }); break;
    }
}

/** Create a Meet-style meeting (also
blocks calendars) and invite everyone by
in-app + email + SMS. */
async meets(companyId: string, hostId:
string, kind: string, title: string,
startsAt: string, durationMin: number,
attendeeIds: string[], ctx: any = {}) {
    const roomId = rid(10), link =
`${BASE_URL()}/meet/${roomId}`;
    const m = await
    this.db.meetings.save(this.db.meetings.crea
    te({ companyId, hostId, kind, title,
    startsAt, durationMin, roomId, link,
    attendeeIds: [...new Set([hostId,
    ...attendeeIds])], ctx, status: 'SCHEDULED'
    }));
    await this.n.send(m.attendeeIds, {
    title: `Meeting: ${title}`, body: `${new
    Date(startsAt).toLocaleString()} `

```

```

    ${durationMin} min · Join: ${link}`, type:
    'MEETING', link: `/meet/${roomId}`, sms:
    true });
    return m;
  }
}

/* WebSocket gateway: live notifications +
WebRTC signalling for the built-in
"Nerkanal Meet" */
@WebSocketGateway({ cors: { origin: '*' }
})
class AppGateway implements
OnGatewayConnection, OnGatewayDisconnect {
  @WebSocketServer() server: Server;
  constructor(private jwt: JwtService,
private db: Db) {}
  afterInit(s: Server) { Live.io = s; }
  handleConnection(c: Socket) {
    try { const p: any =
this.jwt.verify((c.handshake.auth?.token ||
c.handshake.query?.token) as string); (c as
any).uid = p.sub; c.join(`user:${p.sub}`);
}
    catch { c.disconnect(true); }
  }
  async handleDisconnect(c: Socket) { const
r = (c as any).room; if (r)
c.to(r).emit('peer-left', { id: c.id }); }
  @SubscribeMessage('join') async join(c:
Socket, d: { roomId: string; name: string
}) {
    const m = await
this.db.meetings.findOneBy({ roomId:

```

```

d.roomId });
    if (!m || !m.attendeeIds.includes((c as
any).uid)) return c.emit('denied', 'You are
not invited to this meeting');
    const room = `room:${d.roomId}`; (c as
any).room = room;
    const peers = [...
(this.server.sockets.adapter.rooms.get(room
) || [])].map(id => ({ id, name:
(this.server.sockets.sockets.get(id) as
any)?.dname }));
    (c as any).dname = d.name;
c.join(room); c.emit('peers', peers);
c.to(room).emit('peer-joined', { id: c.id,
name: d.name });
    if (m.status === 'SCHEDULED') {
m.status = 'LIVE'; await
this.db.meetings.save(m); }
    }
    @SubscribeMessage('signal') signal(c:
Socket, d: { to: string; data: any }) {
this.server.to(d.to).emit('signal', { from:
c.id, data: d.data }); }
    @SubscribeMessage('chat') chat(c: Socket,
d: { text: string }) { const r = (c as
any).room; if (r)
this.server.to(r).emit('chat', { from: (c
as any).dname, text: d.text, at: now() });
    }
}

/* External job aggregator (open APIs).
LinkedIn has no public jobs API – use a
licensed aggregator (JSearch/RapidAPI

```

```

covers LinkedIn, Indeed, Glassdoor...). */
@Injectable() class ExternalJobs {
  private cache = new Map<string, { at:
number; rows: any[] }>();
  constructor(private db: Db) {}
  async search(q: string) {
    const key = (q ||
'').toLowerCase().trim(), hit =
this.cache.get(key);
    if (hit && Date.now() - hit.at < 10 *
60e3) return hit.rows;
    const rows: any[] = [];
    try {
      const r = await
fetch(`https://remotive.com/api/remote-
jobs?search=${encodeURIComponent(q ||
'')}&limit=20`).then(x => x.json());
      for (const j of r.jobs || [])
rows.push({ externalId: 'remotive-' + j.id,
title: j.title, companyName:
j.company_name, location:
j.candidate_required_location, applyUrl:
j.url, description: (j.description ||
'').replace(/<[^>]+>/g, ' ').slice(0,
1500), skills: j.tags || [], via:
'Remotive' });
    } catch (e) { log.warn('remotive: ' +
e.message); }
    if (ENV('RAPIDAPI_KEY')) try {
      const r = await
fetch(`https://jsearch.p.rapidapi.com/searc
h?query=${encodeURIComponent(q ||
'jobs')}&num_pages=1`, { headers: { 'X-
RapidAPI-Key': ENV('RAPIDAPI_KEY'), 'X-

```

```

RapidAPI-Host': 'jsearch.p.rapidapi.com' }
}).then(x => x.json());
    for (const j of r.data || [])
rows.push({ externalId: 'jsearch-' +
j.job_id, title: j.job_title, companyName:
j.employer_name, location: [j.job_city,
j.job_country].filter(Boolean).join(', '),
applyUrl: j.job_apply_link, description:
(j.job_description || '').slice(0, 1500),
skills: j.job_required_skills || [], via:
j.job_publisher });
    } catch (e) { log.warn('jsearch: ' +
e.message); }
    for (const x of rows) { // place on our
portal (upsert) with the apply link
        const ex = await
this.db.jobs.findOneBy({ externalId:
x.externalId });
        if (!ex) await
this.db.jobs.save(this.db.jobs.create({
...x, source: 'EXTERNAL', status: 'OPEN'
})));
    }
    this.cache.set(key, { at: Date.now(),
rows }); return rows;
}
}

/* Video-intro AI. Plug a real provider via
VIDEO_AI_URL (must return {faceDetected,
communication, confidence, passion} 0-100).
Without it, a clearly-flagged DEMO
scorer runs so the flow is testable. */
@Injectable() class VideoAI {

```

```

    async analyze(file: string, size: number)
    {
        if (ENV('VIDEO_AI_URL')) {
            const r: any = await
            fetch(ENV('VIDEO_AI_URL'), { method:
            'POST', headers: { 'Content-Type':
            'application/json', Authorization: `Bearer
            ${ENV('VIDEO_AI_KEY')}` }, body:
            JSON.stringify({ url:
            `${BASE_URL()}/files/${path.basename(file)}
            ` }) }).then(x => x.json());
            const overall =
            Math.round((r.communication + r.confidence
            + r.passion) / 3);
            return { faceVerified:
            !!r.faceDetected, communication:
            r.communication, confidence: r.confidence,
            passion: r.passion, overall, demo: false };
        }
        const h = parseInt(sha(file +
        size).slice(0, 6), 16), s = (k: number) =>
        55 + ((h >> k) % 40);
        const c = s(1), f = s(5), p = s(9);
        return { faceVerified: size > 1000,
        communication: c, confidence: f, passion:
        p, overall: Math.round((c + f + p) / 3),
        demo: true };
    }
}

const upload = FileInterceptor('file', {
storage: diskStorage({ destination: (_r,
_f, cb) => { fs.mkdirSync('uploads', {
recursive: true }); cb(null, 'uploads'); },
filename: (_r, f, cb) => cb(null, rid(12) +

```

```

path.extname(f.originalname)) })), limits: {
  fileSize: 200 * 1024 * 1024 } });

/** Thin Claude/LLM helper (Messages API).
Used by mock interviews and Jarvis. Returns
null when no key is configured. */
async function llm(system: string, user:
string, max = 800): Promise<string | null>
{
  if (!ENV('ANTHROPIC_API_KEY')) return
  null;
  const r: any = await
  fetch('https://api.anthropic.com/v1/message
s', { method: 'POST', headers: { 'x-api-
key': ENV('ANTHROPIC_API_KEY'), 'anthropic-
version': '2023-06-01', 'content-type':
'application/json' }, body:
JSON.stringify({ model: ENV('JARVIS_MODEL',
'claude-sonnet-5-5'), max_tokens: max,
system, messages: [{ role: 'user', content:
user }] }) }).then(x => x.json());
  return (r.content || []).filter((b: any)
=> b.type === 'text').map((b: any) =>
b.text).join('\n') || null;
}
const DEF_SETTINGS = {
  probationDays: 90, shiftStart: '09:30',
  graceMin: 10, weekOff: [0], geofences: []
as any[], faceThreshold: 0.8, moodBot:
true,
  ratePerKm: 8, claimLimit: 5000,
  idleMinutes: 5, locationOffEscalateMin: 30,
  requiredDocs: ['ID_PROOF', 'ADDRESS_PROOF',
'EDUCATION', 'PREV_EMPLOYMENT', 'PHOTO'],

```

```

    series: { EMP: { prefix: 'NK', next: 1 },
CON: { prefix: 'NKC', next: 1 } } as any,
statutory: { pf: true, esi: true, pt: true,
lwf: true, pfCeiling: 15000, bonusCeiling:
7000, bonusPct: 8.33 },
};
const cfg = (c: Company) => ({
...DEF_SETTINGS, ...(c?.settings || {}) });
const PLAN = { FREE: { jobs: 2, pool: false
}, PREMIUM: { jobs: 5, pool: true } } as
any;
const QBANK = ['Tell me about yourself.',
'Why do you want this role and what excites
you about it?', 'Describe a challenging
situation you handled and what the result
was.', 'What are your key strengths, and
one area you are improving?', 'Tell me
about a conflict in a team and how you
resolved it.', 'Where do you see yourself
in three years?', 'Why should we hire you
over other candidates?'];
const MOODS: Record<string, number> = {
EXCITED: 5, GOOD: 4, OKAY: 3, STRESSED: 2,
TIRED: 2 };

/* ===== §1 CANDIDATE
PORTAL (free for candidates)
===== */
import { IsEmail, MinLength, IsString,
IsOptional } from 'class-validator';
class RegisterDto { @IsEmail() email:
string; @MinLength(8) password: string;
@IsString() name: string; @IsOptional()
dob?: string; @IsOptional() zodiac?:

```

```

string; @IsOptional() phone?: string; }
class LoginDto { @IsString() email: string;
@IsString() password: string; }

@Ctl('auth') class AuthCtl {
  constructor(private db: Db, private jwt:
JwtService, private n: Notifier) {}
  private async otp(u: User) {
    const code =
String(crypto.randomInt(100000, 999999));
u.otp = sha(code + u.email); u.otpExp = new
Date(Date.now() + 10 * 60e3).toISOString();
await this.db.users.save(u);
    await this.n.send(u.id, { title: 'Your
Nerkanal verification code', body: `Code:
${code} (valid 10 minutes)`, type: 'AUTH'
});
    return ENV('NODE_ENV') === 'production'
? undefined : code;
  }
  private token(u: User) { return
this.jwt.sign({ sub: u.id, role: u.role,
companyId: u.companyId }); }
  @Public() @Post('register') async
register(@Body() b: RegisterDto) {
    if (await this.db.users.findOneBy({
email: b.email.toLowerCase() })) throw new
ConflictException('Email already
registered');
    const sign = b.zodiac || zodiac(b.dob);
    const u = await
this.db.users.save(this.db.users.create({
email: b.email.toLowerCase(), passwordHash:
await bcrypt.hash(b.password, 10), name:

```

```

b.name, dob: b.dob, phone: b.phone, zodiac:
sign, role: 'CANDIDATE', kind: 'CANDIDATE',
profile: {}, docs: [], profilePct: 0 }));
    u.profilePct = candPct(u); await
this.db.users.save(u);
    return { message: 'Verification code
sent to your email', zodiac: sign, devOtp:
await this.otp(u) };
}
/** Buyers (companies) choose which
modules to buy at sign-up. */
@Public() @Post('register-company') async
registerCompany(@Body() b: any) {
    if (!b.email || !b.password ||
!b.companyName) throw new
BadRequestException('companyName, email,
password required');
    if (await this.db.users.findOneBy({
email: b.email.toLowerCase() })) throw new
ConflictException('Email already
registered');
    const modules = (b.modules ||
[]).filter((m: string) =>
MODULE_KEYS.includes(m));
    const c = await
this.db.companies.save(this.db.companies.cr
eate({ name: b.companyName, state: b.state,
country: b.country || 'IN', modules,
atsPlan: b.atsPlan === 'PREMIUM' ?
'PREMIUM' : 'FREE', deviceKey: rid(24),
settings: {} }));
    const u = await
this.db.users.save(this.db.users.create({
email: b.email.toLowerCase(), passwordHash:

```

```

await bcrypt.hash(b.password, 10), name:
b.name || 'Super Admin', role:
'SUPER_ADMIN', kind: 'STAFF', companyId:
c.id, profile: {}, docs: [] }));
    return { message: 'Company created.
Verify the code sent to your email.',
companyId: c.id, devOtp: await this.otp(u)
};
}
@Public() @Post('verify') async
verify(@Body() b: { email: string; otp:
string }) {
    const u = await
this.db.users.findOneBy({ email: (b.email
|| '').toLowerCase() });
    if (!u || !u.otp || u.otp !== sha(b.otp
+ u.email) || new Date(u.otpExp) < new
Date()) throw new
BadRequestException('Invalid or expired
code');
    u.verified = true; u.otp = null; await
this.db.users.save(u); return { token:
this.token(u), user: clean(u) };
}
@Public() @Post('resend') async
resend(@Body() b: { email: string }) {
const u = await this.db.users.findOneBy({
email: (b.email || '').toLowerCase() }); if
(u && !u.verified) return { devOtp: await
this.otp(u) }; return { ok: true }; }
@Public() @Post('login') async
login(@Body() b: LoginDto) {
    const id = b.email.toLowerCase();
    const u = (await

```

```

this.db.users.findOneBy({ email: id })) ||
(await this.db.users.findOneBy({ empId:
b.email }));
    if (!u || !(await
bcrypt.compare(b.password, u.passwordHash
|| ''))) throw new
UnauthorizedException('Wrong credentials');
    if (!u.verified) throw new
UnauthorizedException('Email not
verified');
    if (!u.active) throw new
UnauthorizedException('Account
deactivated');
    return { token: this.token(u), user:
clean(u), mustChangePwd: u.mustChangePwd };
}
@Get('me') async me(@Me() u: User) {
    const c = u.companyId ? await
this.db.company(u.companyId) : null;
    return { user: clean(u), company: c &&
{ id: c.id, name: c.name, modules:
c.modules, atsPlan: c.atsPlan } };
}
@Post('change-password') async cp(@Me()
u: User, @Body() b: { oldPassword: string;
newPassword: string }) {
    if (!(await
bcrypt.compare(b.oldPassword,
u.passwordHash))) throw new
UnauthorizedException('Old password
wrong');
    if ((b.newPassword || '').length < 8)
throw new BadRequestException('Min 8
characters');

```

```

        u.passwordHash = await
bcrypt.hash(b.newPassword, 10);
u.mustChangePwd = false; await
this.db.users.save(u); return { ok: true };
    }
}
const PROFILE_FIELDS = ['headline',
'location', 'skills', 'experienceYears',
'education', 'experience', 'resumeUrl',
'expectedCtc', 'noticePeriod',
'languages'];
const candPct = (u: User) => {
    const p = u.profile || {}, f = [u.name,
u.phone, u.dob, ...PROFILE_FIELDS.map(k =>
p[k])];
    return Math.round((f.filter(v =>
(Array.isArray(v) ? v.length : v !==
undefined && v !== null && String(v).trim()
!== '')).length / f.length) * 100);
};

@Ctl('candidate') @Roles('CANDIDATE') class
CandidateCtl {
    constructor(private db: Db, private ai:
VideoAI, private ap: Approvals, private n:
Notifier) {}
    @Get('dashboard') async dash(@Me() u:
User) {
        const apps = await this.db.apps.find({
where: { candidateId: u.id } });
        const meets = (await
this.db.meetings.find({ where: { kind:
'INTERVIEW' } })).filter(m =>
m.attendeeIds.includes(u.id) && m.status

```

```

!= 'DONE');
    return { profilePct: u.profilePct,
canApply: u.profilePct >= 60 && !!u.video,
video: u.video, videoAttemptsLeft:
Math.max(0, 5 - (u.videoAttempts || 0)),
applied: apps.length, upcomingInterviews:
meets.length, horoscope: u.zodiac ?
horoscope(u.zodiac) : null, nextSteps:
[u.profilePct < 60 && 'Complete at least
60% of your profile', !u.video && 'Record
your video self-intro', 'Search & apply for
jobs'].filter(Boolean) };
}
@Get('profile') profile(@Me() u: User) {
return clean(u); }
@Put('profile') async put(@Me() u: User,
@Body() b: any) {
    const { name, phone, dob, zodiac: z,
...rest } = b;
    if (name) u.name = name; if (phone)
u.phone = phone; if (dob) { u.dob = dob;
u.zodiac = z || zodiac(dob); } else if (z)
u.zodiac = z;
    u.profile = { ...(u.profile || {}),
...Object.fromEntries(Object.entries(rest).
filter(([k]) =>
PROFILE_FIELDS.includes(k))) };
    u.profilePct = candPct(u); await
this.db.users.save(u); return { profilePct:
u.profilePct, canApply: u.profilePct >= 60,
user: clean(u) };
}
/** Video self-intro: max 5 attempts;
once finalised the score is frozen for 3

```

```

months. */
    @Post('video') @UseInterceptors(upload)
    async video(@Me() u: User, @UploadedFile()
    f: any) {
        if (u.profilePct < 60) throw new
        ForbiddenException('Complete 60% of your
        profile first');
        if (!f) throw new
        BadRequestException('Attach the video as
        multipart field "file"');
        if (u.videoLockUntil && new
        Date(u.videoLockUntil) > new Date()) throw
        new ForbiddenException(`Your score is
        locked until ${u.videoLockUntil.slice(0,
        10)}`);
        if (u.videoLockUntil) { u.videoAttempts
        = 0; u.videoLockUntil = null; }
        if (u.videoAttempts >= 5) throw new
        ForbiddenException('All 5 attempts used');
        const r = await this.ai.analyze(f.path,
        f.size); u.videoAttempts += 1;
        const rec = await
        this.db.addRec('VIDEO', { ownerId: u.id,
        status: r.faceVerified ? 'VERIFIED' :
        'FACE_NOT_DETECTED', data: { ...r, url:
        `/files/${f.filename}`, attempt:
        u.videoAttempts } });
        if (r.faceVerified) u.video = { ...r,
        url: `/files/${f.filename}`, recordedAt:
        now() };
        if (u.videoAttempts >= 5 && u.video)
        u.videoLockUntil = addMonths(new Date(),
        3);
        await this.db.users.save(u);
    }

```

```

        return { attempt: u.videoAttempts,
        attemptsLeft: 5 - u.videoAttempts, result:
        r, accepted: r.faceVerified, message:
        r.faceVerified ? 'Face verified & scored.
        Finalise to lock your score for 3 months,
        or retake.' : 'Face not detected – please
        retake in good light, facing the camera.'
        };
    }
    @Post('video/finalize') async
    finalize(@Me() u: User) {
        if (!u.video) throw new
        BadRequestException('Record a verified
        video first');
        u.videoLockUntil = addMonths(new
        Date(), 3); await this.db.users.save(u);
        return { locked: true, until:
        u.videoLockUntil, score: u.video };
    }
    @Get('applications') async apps(@Me() u:
    User) {
        const apps = await this.db.apps.find({
        where: { candidateId: u.id }, order: {
        createdAt: 'DESC' } });
        const jobs = await this.db.jobs.find({
        where: { id: In(apps.map(a =>
        a.jobId).concat('-')) } });
        return apps.map(a => ({ ...a, job:
        jobs.find(j => j.id === a.jobId) }));
    }
    @Get('interviews') async interviews(@Me()
    u: User) { return (await
    this.db.meetings.find({ where: { kind:
    'INTERVIEW' }, order: { startsAt: 'ASC' }

```

```

    })).filter(m =>
m.attendeeIds.includes(u.id)); }
    @Get('feedback') async feedback(@Me() u:
User) { return (await
this.db.listRec('FEEDBACK', { status:
'PUBLISHED' })).filter(f =>
f.data.candidateId === u.id).map(f => ({
meetingId: f.refId, rating: f.data.rating,
comments: f.data.comments, publishedAt:
f.data.publishedAt })); }
    @Get('offers') offers(@Me() u: User) {
return this.db.listRec('OFFER', { ownerId:
u.id }); }
    @Post('offers/:id/respond') async
respond(@Me() u: User, @Param('id') id:
string, @Body() b: { accept: boolean;
reason?: string }) {
        const o = await this.db.rec('OFFER',
id); if (o.ownerId !== u.id) throw new
ForbiddenException();
        if (o.status !== 'RELEASED') throw new
BadRequestException(`Offer already
${o.status}`);
        o.status = b.accept ? 'ACCEPTED' :
'DECLINED'; o.data = { ...o.data,
respondedAt: now(), reason: b.reason };
        const app = await
this.db.apps.findOneBy({ id: o.refId });
        if (app) { app.status = b.accept ?
'OFFER_ACCEPTED' : 'OFFER_DECLINED'; await
this.db.apps.save(app); }
        if (b.accept) { o.status =
'PUSHED_TO_HRP'; }
        await this.db.recs.save(o);

```

```

    const hr = await
this.db.ids(o.companyId, b.accept ?
['HRBP', 'HR_MANAGER'] : ['RECRUITER']);
    await this.n.send([...hr,
o.data.recruiterId].filter(Boolean), {
title: `Offer ${b.accept ? 'accepted' :
'declined'}: ${u.name}`, body: b.accept ?
'Candidate pushed to HRBP for onboarding' :
b.reason, type: 'OFFER', link:
`/onboarding`, sms: true });
    return { status: o.status };
  }
  /** Mock interview with Jarvis – generic
questions; feedback with improvement
points. */
  @Post('mock-interview/start') async
mockStart(@Me() u: User, @Body() b: {
role?: string }) {
    let qs = QBANK.slice().sort(() =>
Math.random() - 0.5).slice(0, 5);
    const g = await llm('Return ONLY a JSON
array of 5 concise, commonly asked
interview questions.', `Role: ${b?.role ||
u.profile?.headline || 'general'}`,
400).catch(() => null);
    try { const j = JSON.parse((g ||
'').replace(/```json|```/g, '')); if
(Array.isArray(j) && j.length) qs =
j.slice(0, 5); } catch { /* keep bank */ }
    const m = await this.db.addRec('MOCK',
{ ownerId: u.id, status: 'STARTED', data: {
role: b?.role, questions: qs } });
    return { mockId: m.id, questions: qs };
  }

```

```

@Post('mock-interview/:id/submit') async
mockSubmit(@Me() u: User, @Param('id') id:
string, @Body() b: { answers: string[] }) {
    const m = await this.db.rec('MOCK',
id); if (m.ownerId !== u.id) throw new
ForbiddenException();
    const qs: string[] = m.data.questions,
ans = b.answers || [];
    let fb: any = null;
    const t = await llm('You are an
interview coach. Return ONLY JSON:
{"score":0-100,"perQuestion":
[{"q":"","feedback":"","improve":""}], "topI
mprovements":["",""]}'.',
JSON.stringify(qs.map((q, i) => ({ q, a:
ans[i] || '' }))), 1200).catch(() => null);
    try { fb = JSON.parse((t ||
'').replace(/``json|``/g, '')); } catch {
/* fallback below */ }
    if (!fb) { // heuristic fallback
        const per = qs.map((q, i) => { const
a = (ans[i] || '').trim(), w =
a.split(/\s+/).filter(Boolean).length;
const tips = [];
            if (w < 30) tips.push('Give a
fuller answer (aim for 40-120 words).'); if
(w > 180) tips.push('Be more concise. ');
            if (!/\d/.test(a)) tips.push('Add
measurable results or numbers. '); if (!(i
|my |we )/i.test(a)) tips.push('Use
specific personal examples (STAR
method). ');
            return { q, feedback: tips.length ?
'Needs work' : 'Good structure', improve:

```

```

tips.join(' ') || 'Keep it up.', pts:
Math.max(0, 100 - tips.length * 22) }); });
    fb = { score:
Math.round(per.reduce((s, x) => s + x.pts,
0) / per.length), perQuestion: per,
topImprovements: [...new Set(per.flatMap(x
=> x.improve.split('. ')).filter(x => x &&
x !== 'Keep it up.')).slice(0, 3) ]};
    }
    m.status = 'DONE'; m.data = {
...m.data, answers: ans, feedback: fb };
await this.db.recs.save(m); return fb;
  }
}

```

```

@Ctl('jobs') class JobsCtl {
  constructor(private db: Db, private ext:
ExternalJobs, private n: Notifier) {}
  private gate(u: User) { if (u.role ===
'CANDIDATE' && (u.profilePct < 60 ||
!u.video)) throw new
ForbiddenException('Complete ≥60% profile
and your video self-intro to search & apply
for jobs'); }
  @Get('search') @Roles('CANDIDATE') async
search(@Me() u: User, @Query('q') q = '',
@Query('skills') skills = '') {
    this.gate(u);
    const ql = q.toLowerCase(), sk =
skills.toLowerCase().split(',').map(s =>
s.trim()).filter(Boolean);
    const match = (j: Job) => (!ql ||
`${j.title} ${j.description} ${j.skills ||
[]}.join(' ')`.toLowerCase().includes(ql))

```

```

    && sk.every(s => `${j.title}
    ${j.description} ${j.skills || []}.join('
    ')}'.toLowerCase().includes(s));
    await this.ext.search(q || sk[0] ||
    '');
    const all = await this.db.jobs.find({
    where: { status: 'OPEN' }, order: {
    createdAt: 'DESC' }, take: 500 });
    const internal = all.filter(j =>
    j.source === 'INTERNAL' && match(j)),
    external = all.filter(j => j.source ===
    'EXTERNAL' && match(j)).slice(0, 40);
    return { internal, external, results:
    [...internal, ...external] }; // recruiter-
    posted jobs always first
    }
    @Get('/:id') one(@Param('id') id: string)
    { return this.db.jobs.findOneByOrFail({ id
    }); }
    @Post('/:id/apply') @Roles('CANDIDATE')
    async apply(@Me() u: User, @Param('id') id:
    string) {
        this.gate(u); const j = await
        this.db.jobs.findOneBy({ id }); if (!j ||
        j.status !== 'OPEN') throw new
        NotFoundException('Job not available');
        if (j.source === 'EXTERNAL') { await
        this.db.addRec('EXT_CLICK', { ownerId:
        u.id, refId: j.id, data: { url: j.applyUrl
        } }); return { redirect: j.applyUrl }; }
        if (await this.db.apps.findOneBy({
        jobId: id, candidateId: u.id })) throw new
        ConflictException('Already applied');
        const a = await

```

```

this.db.apps.save(this.db.apps.create({
  companyId: j.companyId, jobId: id,
  candidateId: u.id, status: 'APPLIED',
  timeline: [{ at: now(), status: 'APPLIED'
}], docs: [] }));
    await this.n.send(j.recruiterId, {
  title: `New application: ${j.title}`, body:
  `${u.name} (profile ${u.profilePct}%, video
  ${u.video?.overall}/100)`, type: 'ATS',
  link: `/ats/jobs/${id}` });
    return a;
  }
}

/* ===== $2 RECRUITER
ATS (Feature: ats)
===== */
const offerHtml = (c: Company, cand: User,
o: any) => `<div style="font-
family:Arial;max-width:720px;margin:auto">
<h2 style="color:#0B1F4B">${c.name}</h2>
<p>Date: ${ymd()}</p><p>Dear ${cand.name},
</p><p>We are delighted to offer you the
position of <b>${o.designation}</b> with an
annual CTC of <b>₹
${Number(o.ctc).toLocaleString('en-IN')}
</b>. Your expected date of joining is
<b>${o.doj}</b>.</p><p>This offer is valid
until ${o.expiresOn}. Please accept or
decline it from your Nerkanal candidate
portal.</p><p>Regards,<br/>HR Team,
${c.name}</p></div>`;
@Ctl('ats') @Feat('ats') class AtsCtl {
  constructor(private db: Db, private ap:

```

```

Approvals, private n: Notifier) {}
  /* Step 1: manpower requisition raised by
  department manager/head → HR manager
  approves → assigned to recruiter */
  @Post('requisitions') @Roles('DEPT_HEAD',
  'RM', 'HR_MANAGER') async req(@Me() u:
  User, @Body() b: { title: string;
  headcount: number; department?: string;
  skills?: string[]; justification?: string;
  budget?: number }) {
    if (!b.title || !b.headcount) throw new
  BadRequestException('title & headcount
  required');
    return this.ap.create('REQUISITION', u,
  { ...b, recruiterId: null },
  ['HR_MANAGER'], `Manpower request:
  ${b.title} ×${b.headcount}`);
  }
  @Get('requisitions') async reqs(@Me() u:
  User) {
    const all = await
  this.db.approvals.find({ where: {
  companyId: u.companyId, type: 'REQUISITION'
  }, order: { createdAt: 'DESC' } });
    return all.filter(a => [...HRS,
  'ADMIN', 'SUPER_ADMIN'].includes(u.role) ||
  a.requesterId === u.id ||
  a.payload.recruiterId === u.id);
  }
  @Post('requisitions/:id/assign')
  @Roles('HR_MANAGER', 'HR', 'HRBP') async
  assign(@Me() u: User, @Param('id') id:
  string, @Body() b: { recruiterId: string })
  {

```

```

        const a = await
this.db.approvals.findOneBy({ id,
companyId: u.companyId, type: 'REQUISITION'
}); if (!a) throw new NotFoundException();
    if (a.status !== 'APPROVED') throw new
BadRequestException('Requisition must be
approved first');
    const r = await
this.db.user(b.recruiterId); if (r.role !==
'RECRUITER') throw new
BadRequestException('Not a recruiter');
    a.payload = { ...a.payload,
recruiterId: r.id }; await
this.db.approvals.save(a);
    await this.n.send([r.id,
a.requesterId], { title: `Requisition
assigned: ${a.payload.title}`, body:
`Recruiter: ${r.name}`, type: 'ATS', link:
`/ats/requisitions/${id}` }); return a;
    }
    @Get('recruiters') @Roles('HR_MANAGER',
'HR', 'HRBP') recs(@Me() u: User) { return
this.db.byRole(u.companyId,
['RECRUITER']).then(x => x.map(clean)); }
    /* Step 2: job post (Free: 2 open posts ·
Premium: 5 + candidate pool) */
    @Post('jobs') @Roles('RECRUITER',
'HR_MANAGER') async job(@Me() u: User,
@Body() b: { requisitionId: string; title?:
string; description: string; skills?:
string[]; location?: string }) {
        const c = await
this.db.company(u.companyId), plan =
PLAN[c.atsPlan] || PLAN.FREE;

```

```

    const r = await
    this.db.approvals.findOneBy({ id:
    b.requisitionId, companyId: u.companyId,
    type: 'REQUISITION' });
    if (!r || r.status !== 'APPROVED')
    throw new BadRequestException('Approved
    requisition required');
    if (r.payload.recruiterId !== u.id &&
    u.role !== 'HR_MANAGER') throw new
    ForbiddenException('Requisition is not
    assigned to you');
    const open = await this.db.jobs.count({
    where: { companyId: u.companyId, source:
    'INTERNAL', status: 'OPEN' } });
    if (open >= plan.jobs) throw new
    ForbiddenException(`${c.atsPlan} plan
    allows ${plan.jobs} open job posts.
    ${c.atsPlan === 'FREE' ? 'Upgrade to
    Premium for 5 posts + candidate pool.' :
    'Close one first.'}`);
    const j = await
    this.db.jobs.save(this.db.jobs.create({
    companyId: u.companyId, reqId: r.id,
    recruiterId: u.id, title: b.title ||
    r.payload.title, description:
    b.description, skills: b.skills ||
    r.payload.skills || [], location:
    b.location, source: 'INTERNAL',
    companyName: c.name, status: 'OPEN' }));
    const url =
    `${BASE_URL()}/jobs/${j.id}`;
    return { job: j, share: { link: url,
    linkedin:
    `https://www.linkedin.com/sharing/share-

```

```

offsite/?url=${encodeURIComponent(url)}`,
whatsapp: `https://wa.me/?
text=${encodeURIComponent(j.title + ' ' +
url)}`, mail: `mailto:?
subject=${encodeURIComponent(j.title)}&body
=${encodeURIComponent(url)}` } } };
    }
    @Get('jobs') jobs(@Me() u: User) { return
this.db.jobs.find({ where: { companyId:
u.companyId }, order: { createdAt: 'DESC' }
}); }
    @Put('jobs/:id/close') async close(@Me()
u: User, @Param('id') id: string) { const j
= await this.db.jobs.findOneByOrFail({ id,
companyId: u.companyId }); j.status =
'CLOSED'; return this.db.jobs.save(j); }
    @Get('jobs/:id/applications') async
applications(@Me() u: User, @Param('id')
id: string) {
        const apps = await this.db.apps.find({
where: { jobId: id, companyId: u.companyId
} });
        const us = await this.db.users.find({
where: { id: In(apps.map(a =>
a.candidateId).concat('-')) } });
        return apps.map(a => ({ ...a,
candidate: clean(us.find(x => x.id ===
a.candidateId)) }));
    }
    @Get('pool') async pool(@Me() u: User,
@Query('q') q = '') {
        const c = await
this.db.company(u.companyId); if
(!PLAN[c.atsPlan]?.pool) throw new

```

```

ForbiddenException('Candidate pool is a
Premium feature');
    const all = await this.db.users.find({
where: { role: 'CANDIDATE', verified: true,
active: true } });
    return all.filter(x => x.profilePct >=
60 && x.video && `${x.name}
${JSON.stringify(x.profile)}`.toLowerCase()
.includes(q.toLowerCase())).map(x => ({ id:
x.id, name: x.name, headline:
x.profile?.headline, skills:
x.profile?.skills, experienceYears:
x.profile?.experienceYears, videoScore:
x.video?.overall, profilePct: x.profilePct
}));
    }
    @Post('applications/:id/screen') async
screen(@Me() u: User, @Param('id') id:
string, @Body() b: { decision: 'SHORTLIST'
| 'REJECT'; notes?: string }) {
        const a = await
this.db.apps.findOneByOrFail({ id,
companyId: u.companyId });
        a.status = b.decision === 'SHORTLIST' ?
'SHORTLISTED' : 'REJECTED'; a.notes =
b.notes; a.timeline = [...(a.timeline ||
[])], { at: now(), status: a.status, by:
u.name }]; await this.db.apps.save(a);
        await this.n.send(a.candidateId, {
title: `Application update`, body: `Your
application is now: ${a.status}`, type:
'ATS', link: '/candidate/applications' });
return a;
    }

```

```

/* Step 3: schedule interview inside
Nerkanal Meet; link goes to candidate,
recruiter & panel by mail + SMS + in-app */
@Post('interviews') async schedule(@Me()
u: User, @Body() b: { applicationId:
string; startsAt: string; durationMin?:
number; panelIds: string[]; round?: string
}) {
    const a = await
this.db.apps.findOneByOrFail({ id:
b.applicationId, companyId: u.companyId }),
cand = await this.db.user(a.candidateId), j
= await this.db.jobs.findOneByOrFail({ id:
a.jobId });
    const m = await
this.ap.meets(u.companyId, u.id,
'INTERVIEW', `Interview: ${cand.name} -
${j.title}${b.round ? ' (' + b.round + ')'
: ''}`, b.startsAt, b.durationMin || 45,
[a.candidateId, ...(b.panelIds || [])], {
applicationId: a.id, candidateId:
a.candidateId, jobId: j.id });
    a.status = 'INTERVIEW'; a.timeline =
[...(a.timeline || []), { at: now(),
status: 'INTERVIEW', meetingId: m.id }];
await this.db.apps.save(a); return m;
}
@Get('interviews') async interviews(@Me()
u: User) { return (await
this.db.meetings.find({ where: { companyId:
u.companyId, kind: 'INTERVIEW' }, order: {
startsAt: 'DESC' } })).filter(m => u.role
=== 'RECRUITER' || HRS.includes(u.role) ?
true : m.attendeeIds.includes(u.id)); }

```

```

    @Post('interviews/:id/feedback') async
    feedback(@Me() u: User, @Param('id') id:
    string, @Body() b: { rating: number;
    comments: string; recommend: boolean }) {
        const m = await
    this.db.meetings.findOneByOrFail({ id,
    companyId: u.companyId }); if
    (!m.attendeeIds.includes(u.id)) throw new
    ForbiddenException('You are not on this
    panel');
        const f = await
    this.db.addRec('FEEDBACK', { companyId:
    u.companyId, ownerId: u.id, refId: id,
    status: 'SUBMITTED', data: { ...b,
    panelName: u.name, applicationId:
    m.ctx?.applicationId, candidateId:
    m.ctx?.candidateId } });
        await this.n.send(m.hostId, { title:
    `Panel feedback submitted by ${u.name}`,
    type: 'ATS', link: `/ats/interviews/${id}`
    }); return f;
    }
    @Get('interviews/:id/feedback')
    @Roles('RECRUITER', 'HR_MANAGER', 'HR',
    'HRBP') fb(@Param('id') id: string) {
    return this.db.listRec('FEEDBACK', { refId:
    id }); }
    @Post('feedback/:id/publish')
    @Roles('RECRUITER', 'HR_MANAGER') async
    publish(@Param('id') id: string, @Body() b:
    { comments?: string }) {
        const f = await this.db.rec('FEEDBACK',
    id); f.status = 'PUBLISHED'; f.data = {
    ...f.data, publishedAt: now(), ...

```

```

(b?.comments ? { comments: b.comments } :
{}), panelName: undefined }); await
this.db.recs.save(f);
    await this.n.send(f.data.candidateId, {
title: 'Interview feedback available',
type: 'ATS', link: '/candidate/feedback',
sms: true }); return f;
    }
    /* Step 4: documentation → verification
→ offer → accepted offers are pushed to
HRBP */
    @Post('applications/:id/documents') async
docs(@Me() u: User, @Param('id') id:
string, @Body() b: { docs: { name: string;
url: string }[] }) { const a = await
this.db.apps.findOneByOrFail({ id,
companyId: u.companyId }); a.docs = [...
(a.docs || []), ...b.docs.map(d => ({ ...d,
verified: false }))]]; return
this.db.apps.save(a); }
    @Post('applications/:id/verify-docs')
@Roles('RECRUITER', 'HR_MANAGER') async
verifyDocs(@Me() u: User, @Param('id') id:
string) { const a = await
this.db.apps.findOneByOrFail({ id,
companyId: u.companyId }); if
(!a.docs?.length) throw new
BadRequestException('No documents'); a.docs
= a.docs.map(d => ({ ...d, verified: true,
verifiedBy: u.name })); return
this.db.apps.save(a); }
    @Post('offers') @Roles('RECRUITER',
'HR_MANAGER') async offer(@Me() u: User,
@Body() b: { applicationId: string;

```

```

    designation: string; ctc: number; doj:
    string; expiresOn?: string }) {
        const a = await
    this.db.apps.findOneByOrFail({ id:
    b.applicationId, companyId: u.companyId });
        if (!a.docs?.length || a.docs.some(d =>
    !d.verified)) throw new
    BadRequestException('Verify candidate
    documents before releasing the offer');
        const c = await
    this.db.company(u.companyId), cand = await
    this.db.user(a.candidateId), o = { ...b,
    expiresOn: b.expiresOn || addDays(new
    Date(), 7), recruiterId: u.id };
        const rec = await
    this.db.addRec('OFFER', { companyId:
    u.companyId, ownerId: cand.id, refId: a.id,
    status: 'RELEASED', data: { ...o, html:
    offerHtml(c, cand, o), candidateName:
    cand.name } });
        a.status = 'OFFERED'; await
    this.db.apps.save(a);
        await this.n.send(cand.id, { title:
    `Offer from ${c.name}`, body:
    `${b.designation} – respond by
    ${o.expiresOn}`, type: 'OFFER', link:
    '/candidate/offers', sms: true }); return
    rec;
    }
    @Get('offers') offers(@Me() u: User) {
    return this.db.listRec('OFFER', {
    companyId: u.companyId }); }
    }

```

```

/* ===== §3
ONBOARDING (Feature: onboarding)
===== */

const apptLetter = (c: Company, u: User, o:
any) => `
```

```

req.length; const filled = [...base,
...f].filter(v => v !== undefined && v !==
null && String(v).trim() !== '').length +
req.filter(r => got.includes(r)).length;
return Math.round((filled / total) *
100);
};
@Ctl('onboarding') @Feat('onboarding')
class OnboardingCtl {
  constructor(private db: Db, private ap:
Approvals, private n: Notifier) {}
  @Get('pending') @Roles('HR', 'HRBP',
'HR_MANAGER') async pending(@Me() u: User)
{ return this.db.listRec('OFFER', {
companyId: u.companyId, status:
'PUSHED_TO_HRBP' }); }
  /** On the joining day HR triggers
"Joined": candidate ID → employee ID in
the right series, appointment letter, new
login. */
  @Post('join') @Roles('HR', 'HRBP',
'HR_MANAGER') async join(@Me() hr: User,
@Body() b: { offerId: string; doj?: string;
rmId?: string; hrbpId?: string;
department?: string; series?: string }) {
    const o = await this.db.rec('OFFER',
b.offerId); if (o.status !==
'PUSHED_TO_HRBP') throw new
BadRequestException('Offer is not
accepted/pushed');
    const c = await
this.db.company(hr.companyId), s = cfg(c),
series = b.series || 'EMP', ser =
s.series[series]; if (!ser) throw new

```

```

BadRequestException('Unknown ID series');
    const u = await
this.db.user(o.ownerId);
    u.candidateId = u.id; u.empId =
`${ser.prefix}${String(ser.next).padStart(4
, '0')}`; ser.next += 1; c.settings = {
...s, series: { ...s.series, [series]: ser
} }; await this.db.companies.save(c);
    u.companyId = c.id; u.kind =
'EMPLOYEE'; u.role = 'EMPLOYEE';
u.designation = o.data.designation;
u.department = b.department; u.doj = b.doj
|| o.data.doj; u.rmId = b.rmId; u.hrbpId =
b.hrbpId || hr.id;
    u.probationEnd = addDays(u.doj,
s.probationDays); u.probationState = 'ON';
u.onboarding = 'PENDING'; u.mustChangePwd =
true;
    const temp = rid(10) + '#1';
u.passwordHash = await bcrypt.hash(temp,
10); await this.db.users.save(u);
    const letter = await
this.db.addRec('LETTER', { companyId: c.id,
ownerId: u.id, status: 'SENT', data: {
kind: 'APPOINTMENT', html: apptLetter(c, u,
o.data) } });
    o.status = 'JOINED'; await
this.db.recs.save(o);
    const a = await
this.db.apps.findOneBy({ id: o.refId }); if
(a) { a.status = 'HIRED'; await
this.db.apps.save(a); }
    const lv = await
this.db.leaveTypes.find({ where: {

```

```

    companyId: c.id } }); const year =
String(new Date().getFullYear());
    for (const t of lv) if (t.quota) await
this.db.leaveBal.save(this.db.leaveBal.crea
te({ userId: u.id, code: t.code, year, bal:
t.quota }));
    await this.n.send(u.id, { title:
`Welcome to ${c.name}! Your Employee ID is
${u.empId}`, body: `Login with ${u.empId}
(or your email) and temporary password:
${temp}. Appointment letter attached in
portal.`, type: 'ONBOARD', link:
'/onboarding/me', sms: true });
    return { empId: u.empId, tempPassword:
temp, appointmentLetter: letter.data.html,
probationEnd: u.probationEnd };
  }
  @Get('me') async me(@Me() u: User) {
    const c = await
this.db.company(u.companyId), req =
cfg(c).requiredDocs;
    const pols = await
this.db.listRec('POLICY', { companyId:
u.companyId, status: 'PUBLISHED' }), acks =
await this.db.listRec('ACK', { ownerId:
u.id });
    const pending = pols.filter(p =>
!acks.some(a => a.refId === p.id)).map(p =>
({ id: p.id, title: p.data.title }));
    return { empId: u.empId, onboarding:
u.onboarding, profilePct: empPct(u, req),
requiredDocs: req, uploaded: (u.docs ||
[]).map((d: any) => d.type),
pendingPolicies: pending, letters: await

```

```

this.db.listRec('LETTER', { ownerId: u.id
}) });
}
@Put('profile') async profile(@Me() u:
User, @Body() b: any) {
    const c = await
this.db.company(u.companyId); u.profile = {
...(u.profile || {}),
...Object.fromEntries(Object.entries(b).fil
ter(([k]) => EMP_FIELDS.includes(k))) };
    if (b.phone) u.phone = b.phone;
u.profilePct = empPct(u,
cfg(c).requiredDocs); await
this.db.users.save(u); return { profilePct:
u.profilePct };
}
@Post('documents') async doc(@Me() u:
User, @Body() b: { type: string; url:
string }) { const c = await
this.db.company(u.companyId); u.docs = [...
(u.docs || []).filter((d: any) => d.type
!== b.type), { type: b.type, url: b.url,
at: now() }]; u.profilePct = empPct(u,
cfg(c).requiredDocs); await
this.db.users.save(u); return { profilePct:
u.profilePct }; }
@Post('complete') async complete(@Me() u:
User) {
    const m: any = await this.me(u); if
(m.profilePct < 100) throw new
BadRequestException(`Profile is
${m.profilePct}% – complete 100%`);
    if (m.pendingPolicies.length) throw new
BadRequestException('Acknowledge all

```

```

policies with digital signature first');
    u.onboarding = 'DONE'; await
this.db.users.save(u); await
this.n.send(await this.db.ids(u.companyId,
HRS), { title: `Onboarding completed:
${u.name} (${u.empId})`, type: 'ONBOARD'
}); return { onboarding: 'DONE' };
}

/* Probation – HR defines period in
Settings; 2-day advance alert via cron; HR
confirms / extends / separates */
@Get('probation') @Roles('HR', 'HRBP',
'HR_MANAGER') async prob(@Me() u: User) {
return (await
this.db.employees(u.companyId)).filter(e =>
e.probationState === 'ON').map(e => ({ id:
e.id, name: e.name, empId: e.empId,
probationEnd: e.probationEnd, daysLeft:
diffDays(ymd(), e.probationEnd) })); }

@Post('probation/:userId') @Roles('HR',
'HRBP', 'HR_MANAGER') async decide(@Me()
hr: User, @Param('userId') id: string,
@Body() b: { action: 'CONFIRM' | 'EXTEND' |
'SEPARATE'; days?: number; reason?: string
}) {
    const e = await this.db.user(id);
    if (b.action === 'CONFIRM') {
e.probationState = 'CONFIRMED'; }
    else if (b.action === 'EXTEND') {
e.probationEnd = addDays(e.probationEnd,
b.days || 30); }
    else { e.probationState = 'SEPARATING';
await this.ap.create('TERMINATION', e, {
reason: b.reason || 'Probation not cleared'

```

```

}, ['HR_MANAGER'], `Separation after
probation: ${e.name}`); }

    await this.db.users.save(e); await
this.n.send(e.id, { title: `Probation
update: ${b.action}`, type: 'ONBOARD', sms:
true }); return { probationState:
e.probationState, probationEnd:
e.probationEnd };
    }
}

/* ===== $4
ATTENDANCE (Feature: attendance)
===== */
@Ctl('attendance') @Feat('attendance')
class AttendanceCtl {
    constructor(private db: Db, private ap:
Approvals, private n: Notifier) {}
    @Get('mood-options') moods() { return
Object.keys(MOODS).map(k => ({ code: k,
emoji: { EXCITED: '😄', GOOD: '😊', OKAY:
'😐', STRESSED: '😓', TIRED: '😫' }[k],
label: k[0] + k.slice(1).toLowerCase() }));
}

    /** Core punch used by app / face / geo /
device. Late, WFH, out-of-geofence become
RM requests with lat/lng + Google Maps
preview. */
    async register(u: User, c: Company, o: {
type: 'IN' | 'OUT'; mode: string; at?:
Date; lat?: number; lng?: number; mood?:
string; wfh?: boolean; reason?: string;
faceScore?: number }) {
        const s = cfg(c), at = o.at || new

```

```

Date(), date = ymd(at), time =
at.toISOString();
    let r = await this.db.att.findOneBy({
userId: u.id, date });
    if (o.type === 'OUT') {
        if (!r?.inAt) throw new
BadRequestException('No punch-in found for
today'); r.outAt = time; await
this.db.att.save(r);
        return { ...r, hours: round2((+new
Date(time) - +new Date(r.inAt)) / 36e5) };
    }
    if (r?.inAt) throw new
ConflictException('Already punched in
today');
    if (o.mode === 'FACE' && (o.faceScore
?? 0) < s.faceThreshold) throw new
BadRequestException('Face not recognised –
try again or use app button');
    let kind: string = null, dist: number =
null;
    if (o.wfh) kind = 'WFH';
    else if (o.mode === 'GEO' ||
(s.geofences.length && o.lat !== undefined
&& ![ 'BIOMETRIC', 'DOOR_FACE',
'CCTV' ].includes(o.mode))) {
        if (o.lat === undefined || o.lng ===
undefined) throw new
BadRequestException('Location required for
geo-fenced attendance');
        if (s.geofences.length) { dist =
Math.min(...s.geofences.map((g: any) =>
hav(o.lat, o.lng, g.lat, g.lng) -
g.radiusM)); if (dist > 0) kind =

```

```

'OUT_OF_GEOFENCE'; }
    }
    const [hh, mm] =
s.shiftStart.split(':').map(Number), lim =
new Date(at); lim.setHours(hh, mm +
s.graceMin, 0, 0);
    if (!kind && at > lim) kind = 'LATE';
    r = r || this.db.att.create({ userId:
u.id, companyId: u.companyId, date });
    Object.assign(r, { inAt: time, mode:
o.mode, lat: o.lat ?? 0, lng: o.lng ?? 0,
mood: o.mood, status: kind || 'PRESENT',
flags: { pending: !!kind } });
    await this.db.att.save(r);
    let request: any = null;
    if (kind) {
        const hasLoc = o.lat !== undefined &&
o.lng !== undefined;
        request = await
this.ap.create('ATT_REQ', u, { date, kind,
toStatus: kind === 'WFH' ? 'WFH' : kind ===
'LATE' ? 'LATE' : 'OUT_OF_GEOFENCE',
reason: o.reason, lat: o.lat, lng: o.lng,
mapUrl: hasLoc ? mapUrl(o.lat, o.lng) :
null, mapPreview: hasLoc ? mapEmbed(o.lat,
o.lng) : null, metersOutside: dist && dist
> 0 ? Math.round(dist) : undefined },
['RM'], `${kind.replace(/_/g, ' ')} request
- ${u.name} (${date})`);
    }
    return { attendance: r, request:
request && { id: request.id, pendingWith:
'RM' }, moodPrompt: s.moodBot && !o.mood };
}

```

```

    @Post('punch') async punch(@Me() u: User,
    @Body() b: any) { return this.register(u,
    await this.db.company(u.companyId), { ...b,
    type: b.type || 'IN', mode: b.mode || 'APP'
    }); }

    @Post('mood') async mood(@Me() u: User,
    @Body() b: { mood: string }) { if
    (!MOODS[b.mood]) throw new
    BadRequestException('Unknown mood'); const
    r = await this.db.att.findOneBy({ userId:
    u.id, date: ymd() }); if (!r) throw new
    BadRequestException('Punch in first');
    r.mood = b.mood; return
    this.db.att.save(r); }

    /** Biometric / door face recognition /
    CCTV devices push punches here using the
    company device key. */
    @Public() @Post('device') async
    device(@Body() b: { deviceKey: string;
    empId: string; ts?: string; mode?: string;
    deviceId?: string }) {
        const c = await
        this.db.companies.findOneBy({ deviceKey:
        b.deviceKey }); if (!c) throw new
        UnauthorizedException('Bad device key');
        const u = await
        this.db.users.findOneBy({ companyId: c.id,
        empId: b.empId }); if (!u) throw new
        NotFoundException('Unknown employee');
        const at = b.ts ? new Date(b.ts) : new
        Date(), r = await this.db.att.findOneBy({
        userId: u.id, date: ymd(at) });
        return this.register(u, c, { type:
        r?.inAt ? 'OUT' : 'IN', mode: b.mode ||

```

```

'BIOMETRIC', at }));
    }
    @Get('mine') mine(@Me() u: User,
@Query('month') m = ym()) { return
this.db.att.find({ where: { userId: u.id,
date: Like(m + '%') }, order: { date: 'ASC'
} }); }
    @Get('team') @Roles('RM', 'HR', 'HRBP',
'HR_MANAGER', 'ADMIN') async team(@Me() u:
User, @Query('date') d = ymd()) {
        const people = HRS.includes(u.role) ||
ADMINS.includes(u.role) ? await
this.db.employees(u.companyId) : await
this.db.teamOf(u.id);
        const rows = await this.db.att.find({
where: { companyId: u.companyId, date: d }
});
        return people.map(p => { const r =
rows.find(x => x.userId === p.id); return {
id: p.id, name: p.name, empId: p.empId,
status: r?.status || 'NOT_MARKED', inAt:
r?.inAt, outAt: r?.outAt, mood: r?.mood,
mode: r?.mode, pendingApproval:
r?.flags?.pending }; });
    }
    @Post('requests') async request(@Me() u:
User, @Body() b: { kind: 'REGULARIZATION' |
'WFH' | 'LATE' | 'GEO'; date: string;
reason: string; inAt?: string; outAt?:
string; lat?: number; lng?: number }) {
        if (!b.reason) throw new
BadRequestException('Reason required');
        const hasLoc = b.lat !== undefined &&
b.lng !== undefined;

```

```

        return this.ap.create('ATT_REQ', u, {
            ...b, toStatus: b.kind === 'WFH' ? 'WFH' :
            'PRESENT', mapUrl: hasLoc ? mapUrl(b.lat,
            b.lng) : null, mapPreview: hasLoc ?
            mapEmbed(b.lat, b.lng) : null }, ['RM'],
            `${b.kind} request - ${u.name}
            (${b.date})`);
    }

    @Get('mood-report') @Roles('RM', 'HR',
    'HRBP', 'HR_MANAGER') async
    moodReport(@Me() u: User, @Query('date') d
    = ymd()) { return moodReport(this.db,
    u.companyId, d, HRS.includes(u.role) ?
    undefined : u.id); }

    /* Show-cause notices (3 days uninformed
    absence → day 4 notice → 7 days no reply
    → termination initiated) */

    @Get('showcause') async sc(@Me() u: User)
    { return this.db.listRec('SHOWCAUSE',
    HRS.includes(u.role) ? { companyId:
    u.companyId } : { ownerId: u.id }); }

    @Post('showcause/:id/respond') async
    scRespond(@Me() u: User, @Param('id') id:
    string, @Body() b: { response: string }) {
        const r = await
        this.db.rec('SHOWCAUSE', id); if (r.ownerId
        !== u.id) throw new ForbiddenException();
        r.status = 'RESPONDED'; r.data = {
        ...r.data, response: b.response,
        respondedAt: now() }; await
        this.db.recs.save(r);

        await this.n.send([u.rmId, ...(await
        this.db.ids(u.companyId, HRS))], { title:
        `Show-cause reply from ${u.name}`, type:

```

```

'DISCIPLINE' }); return r;
    }
}

async function moodReport(db: Db,
companyId: string, date: string, rmId?:
string) {
    const people = rmId ? await
db.teamOf(rmId) : await
db.employees(companyId), rows = await
db.att.find({ where: { companyId, date }
});
    const mine = rows.filter(r =>
people.some(p => p.id === r.userId) &&
r.mood), counts: any = {});
    mine.forEach(r => (counts[r.mood] =
(counts[r.mood] || 0) + 1));
    const avg = mine.length ?
round2(mine.reduce((s, r) => s +
(MOODS[r.mood] || 3), 0) / mine.length) :
null;
    return { date, teamSize: people.length,
responded: mine.length, counts,
enthusiasmScore: avg, concerns:
mine.filter(r => (MOODS[r.mood] || 3) <=
2).map(r => people.find(p => p.id ===
r.userId)?.name) };
}

/* ===== $5 LEAVE
(Feature: leave)
===== */
@Ctl('leave') @Feat('leave') class LeaveCtl
{
    constructor(private db: Db, private ap:

```

```

Approvals) {}

@Get('types') types(@Me() u: User) {
return this.db.leaveTypes.find({ where: {
companyId: u.companyId } })); }

@Post('types') @Roles('HRBP',
'HR_MANAGER', 'HR') async addType(@Me() u:
User, @Body() b: any) { return
this.db.leaveTypes.save(this.db.leaveTypes.
create({ companyId: u.companyId, code:
b.code?.toUpperCase(), name: b.name, quota:
b.quota || 0, carry: !!b.carry, paid:
b.paid !== false, applicableTo:
b.applicableTo || ['ALL'], proofAfterDays:
b.proofAfterDays || 3 })); }

@Put('types/:id') @Roles('HRBP',
'HR_MANAGER', 'HR') async upType(@Me() u:
User, @Param('id') id: string, @Body() b:
any) { const t = await
this.db.leaveTypes.findOneByOrFail({ id,
companyId: u.companyId }); Object.assign(t,
b, { id, companyId: u.companyId }); return
this.db.leaveTypes.save(t); }

@Get('balances') balances(@Me() u: User)
{ return this.db.leaveBal.find({ where: {
userId: u.id, year: String(new
Date().getFullYear()) } })); }

@Post('credit-year') @Roles('HRBP',
'HR_MANAGER', 'HR') async credit(@Me() u:
User) { return creditYear(this.db,
u.companyId); }

@Post('apply') async apply(@Me() u: User,
@Body() b: { code: string; from: string;
to?: string; reason: string; proofUrl?:
string; halfDay?: boolean }) {

```

```

    const c = await
this.db.company(u.companyId), s = cfg(c), t
= await this.db.leaveTypes.findOneBy({
companyId: u.companyId, code: (b.code ||
'').toUpperCase() });
    if (!t) throw new
BadRequestException('Unknown leave type');
const to = b.to || b.from; if (to < b.from)
throw new BadRequestException('"to" is
before "from"');
    if (!t.applicableTo?.some(x => ['ALL',
u.role, u.designation,
u.department].includes(x))) throw new
ForbiddenException(`${t.name} is not
applicable to you`);
    let days = 0; for (let d = b.from; d <=
to; d = addDays(d, 1)) if
(!s.weekOff.includes(P(d).getDay())) days
+= 1; if (b.halfDay) days = 0.5;
    if (!days) throw new
BadRequestException('Selected dates are all
week-offs');
    if (days >= (t.proofAfterDays || 3) &&
!b.proofUrl) throw new
BadRequestException(`Valid proof is
required for leave of ${t.proofAfterDays ||
3} or more days`);
    if (t.paid) { const bal = await
this.db.leaveBal.findOneBy({ userId: u.id,
code: t.code, year:
String(P(b.from).getFullYear()) }); if
(!bal || bal.bal < days) throw new
BadRequestException(`Insufficient ${t.name}
balance (${bal?.bal ?? 0})`); }

```

```

        return this.ap.create('LEAVE', u, {
            code: t.code, from: b.from, to, days,
            reason: b.reason, proofUrl: b.proofUrl,
            paid: t.paid }, ['RM', 'HR', 'ADMIN'],
            `${t.name}: ${b.from}${to !== b.from ? ' → ' + to : ''} (${days}d) - ${u.name}`);
        }

        @Post('compoff') async compoff(@Me() u:
        User, @Body() b: { workedOn: string; days?:
        number; reason: string }) { return
        this.ap.create('COMPOFF', u, { ...b, days:
        b.days || 1 }, ['RM', 'HR'], `Comp-off
        credit: worked ${b.workedOn} - ${u.name}`);
        }

        @Get('mine') async mine(@Me() u: User) {
            return Promise.all((await
            this.db.approvals.find({ where: {
            requesterId: u.id }, order: { createdAt:
            'DESC' } })).filter(a => ['LEAVE',
            'COMPOFF',
            'ATT_REQ'].includes(a.type)).map(a =>
            this.ap.view(a))); }
        }

        async function creditYear(db: Db,
        companyId: string) {
            const year = String(new
            Date().getFullYear()), types = await
            db.leaveTypes.find({ where: { companyId }
            }), emps = await db.employees(companyId);
            let n = 0;
            for (const e of emps) for (const t of
            types) {
                if (!t.quota) continue; const prev =
                await db.leaveBal.findOneBy({ userId: e.id,

```

```

code: t.code, year });
    if (prev) continue; const last = await
db.leaveBal.findOneBy({ userId: e.id, code:
t.code, year: String(+year - 1) });
    await
db.leaveBal.save(db.leaveBal.create({
userId: e.id, code: t.code, year, bal:
t.quota + (t.carry && last ? Math.max(0,
last.bal) : 0) })); n++;
  }
  return { credited: n };
}

/* ===== $6 PAYROLL
(Feature: payroll)
===== */
/** Labour-law rules live in DB (Rec type
LAW) so they can be edited without a
deploy. Seeded values are SAMPLES – verify
against current gazettes. */
const SEED_LAWS = [
  { country: 'IN', kind: 'PF',
employeeRate: 12, employerRate: 12,
epsRate: 8.33, wageCeiling: 15000, note:
'EPF: 12% + 12%; EPS 8.33% on wages up to
₹15,000' },
  { country: 'IN', kind: 'ESI',
employeeRate: 0.75, employerRate: 3.25,
wageCeiling: 21000, note: 'ESI applicable
where monthly gross ≤ ₹21,000' },
  { country: 'IN', kind: 'BONUS', pct:
8.33, wageCeiling: 21000, calcCeiling:
7000, note: 'Payment of Bonus Act: min
8.33%, eligibility wage ≤ ₹21,000' },

```

```
{ country: 'IN', kind: 'GRATUITY', days:
15, perDays: 26, minYears: 5, note:
'Payment of Gratuity Act: 15/26 × last
basic × years (≥5)' },
  { country: 'IN', state: 'Karnataka',
kind: 'PT', mult: 1, slabs: [{ from: 0, to:
24999, amt: 0 }, { from: 25000, to: 1e12,
amt: 200 }], sample: true },
  { country: 'IN', state: 'Maharashtra',
kind: 'PT', mult: 1, slabs: [{ from: 0, to:
7500, amt: 0 }, { from: 7501, to: 10000,
amt: 175 }, { from: 10001, to: 1e12, amt:
200 }], sample: true },
  { country: 'IN', state: 'Telangana',
kind: 'PT', mult: 1, slabs: [{ from: 0, to:
15000, amt: 0 }, { from: 15001, to: 20000,
amt: 150 }, { from: 20001, to: 1e12, amt:
200 }], sample: true },
  { country: 'IN', state: 'Tamil Nadu',
kind: 'PT', mult: 6, months: [3, 9], slabs:
[{ from: 0, to: 21000, amt: 0 }, { from:
21001, to: 30000, amt: 135 }, { from:
30001, to: 45000, amt: 315 }, { from:
45001, to: 60000, amt: 690 }, { from:
60001, to: 75000, amt: 1025 }, { from:
75001, to: 1e12, amt: 1250 }], note: 'Half-
yearly slab on 6× monthly gross, deducted
in Mar & Sep', sample: true },
  { country: 'IN', state: 'Tamil Nadu',
kind: 'MIN_WAGE', monthly: 10000, sample:
true, note: 'SAMPLE minimum wage – replace
with notified rate for the scheduled
employment' },
  { country: 'IN', state: 'Karnataka',
```

```

kind: 'MIN_WAGE', monthly: 12000, sample:
true, note: 'SAMPLE minimum wage – replace
with notified rate' },
  { country: 'IN', state: 'Maharashtra',
kind: 'MIN_WAGE', monthly: 11000, sample:
true, note: 'SAMPLE minimum wage – replace
with notified rate' },
  { country: 'IN', state: 'Telangana',
kind: 'MIN_WAGE', monthly: 11000, sample:
true, note: 'SAMPLE minimum wage – replace
with notified rate' },
];
const lawsFor = async (db: Db, c: Company)
=> (await db.listRec('LAW')).filter(l =>
(l.data.country || 'IN') === (c.country ||
'IN') && (!l.data.state || l.data.state ===
c.state));
const SAMPLE_STRUCT = [
  { code: 'BASIC', name: 'Basic', type:
'EARNING', calc: 'PERCENT', of: 'CTC',
value: 50 },
  { code: 'HRA', name: 'HRA', type:
'EARNING', calc: 'PERCENT', of: 'BASIC',
value: 40 },
  { code: 'CONV', name: 'Conveyance', type:
'EARNING', calc: 'FIXED', value: 1600 },
  { code: 'SPL', name: 'Special Allowance',
type: 'EARNING', calc: 'FORMULA', formula:
'MAX(CTC - BASIC - HRA - CONV, 0)' },
];
function computePay(c: Company, u: User,
structure: any[], ctc: number, paidDays:
number, days: number, month: string, laws:
Rec[]) {

```

```

    const s = cfg(c), st = s.statutory, vars:
Record<string, number> = { CTC: ctc,
PAID_DAYS: paidDays, DAYS: days };
    const earnings: any[] = [], deductions:
any[] = [];
    for (const k of structure) { // any
structure: FIXED value, PERCENT of another
component, or FORMULA
        let amt = k.calc === 'FIXED' ?
Number(k.value) : k.calc === 'PERCENT' ?
((vars[k.of] ?? (() => { throw new
BadRequestException(`${k.code}` refers to
unknown `${k.of}`)); }())) *
Number(k.value)) / 100 :
evalExpr(k.formula, vars);
        vars[k.code] = amt; // formulas always
see the FULL-month component value
        if (k.type === 'EARNING' && k.prorate
!== false) amt = (amt * paidDays) / days;
        amt = round2(amt); (k.type ===
'EARNING' ? earnings : deductions).push({
code: k.code, name: k.name, amount: amt });
    }
    const gross = round2(earnings.reduce((a,
b) => a + b.amount, 0)), basic =
earnings.find(e => e.code ===
'BASIC')?.amount || 0;
    const L = (kind: string) => laws.find(l
=> l.data.kind === kind)?.data, pf =
L('PF'), esi = L('ESI'), pt = L('PT'), lwf
= L('LWF');
    const stat: any = { pfEE: 0, pfER: 0,
epsER: 0, epfER: 0, esiEE: 0, esiER: 0, pt:
0, lwf: 0, lwfER: 0, epfWages: 0, epsWages:

```

```

0 };
  if (st.pf && pf && u.profile?.pfOptOut
    !== true) {
    stat.epfWages = Math.min(basic,
    st.pfCeiling ?? pf.wageCeiling);
    stat.epsWages = Math.min(basic,
    pf.wageCeiling);
    stat.pfEE = Math.round((stat.epfWages *
    pf.employeeRate) / 100); stat.epsER =
    Math.round((stat.epsWages * pf.epsRate) /
    100); stat.pfER = Math.round((stat.epfWages
    * pf.employerRate) / 100); stat.epfER =
    stat.pfER - stat.epsER;
    deductions.push({ code: 'PF', name:
    'Provident Fund (EE)', amount: stat.pfEE,
    statutory: true });
  }
  if (st.esi && esi && gross <=
    esi.wageCeiling && gross > 0) { stat.esiEE
    = Math.ceil((gross * esi.employeeRate) /
    100); stat.esiER = Math.ceil((gross *
    esi.employerRate) / 100); deductions.push({
    code: 'ESI', name: 'ESI (EE)', amount:
    stat.esiEE, statutory: true }); }
  if (st.pt && pt && (!pt.months ||
    pt.months.includes(+month.slice(5)))) {
    const g = gross * (pt.mult || 1), slab =
    pt.slabs.find((x: any) => g >= x.from && g
    <= x.to); stat.pt = slab?.amt || 0; if
    (stat.pt) deductions.push({ code: 'PT',
    name: 'Professional Tax', amount: stat.pt,
    statutory: true }); }
  if (st.lwf && lwf && (!lwf.months ||
    lwf.months.includes(+month.slice(5)))) {

```

```

stat.lwf = lwf.employee || 0; stat.lwfER =
lwf.employer || 0; if (stat.lwf)
deductions.push({ code: 'LWF', name:
'Labour Welfare Fund', amount: stat.lwf,
statutory: true }); }
    const ded = round2(deductions.reduce((a,
b) => a + b.amount, 0));
    return { earnings, deductions, gross,
totalDeductions: ded, net: round2(gross -
ded), statutory: stat, basic, ctc };
}
@Ctl('payroll') @Feat('payroll') class
PayrollCtl {
    constructor(private db: Db, private n:
Notifier) {}
    private async needSigner(u: User) { if (!
[...ADMINS, 'HR',
'HR_MANAGER'].includes(u.role)) throw new
ForbiddenException('Only HR / Admin / Super
Admin can attest'); }
    /** Rules applicable to the company's
state / country – shown to the user when
they configure payroll. */
    @Get('compliance') async compliance(@Me()
u: User) { const c = await
this.db.company(u.companyId); return {
state: c.state, country: c.country, notice:
'Review these statutory rules before
configuring salary structures. Seeded
values marked sample:true must be
verified.', rules: (await lawsFor(this.db,
c)).map(l => ({ id: l.id, ...l.data })) };
}
    @Put('laws') @Roles('ADMIN') async

```

```

law(@Me() u: User, @Body() b: { id?:
string; data: any }) { if (b.id) { const r
= await this.db.rec('LAW', b.id); r.data =
b.data; return this.db.recs.save(r); }
return this.db.addRec('LAW', { data: b.data
}); }

@Get('structures') structures(@Me() u:
User) { return this.db.listRec('STRUCT', {
companyId: u.companyId }); }

@Post('structures') @Roles('HR',
'HR_MANAGER', 'HRBP', 'ADMIN') async
addStruct(@Me() u: User, @Body() b: { name:
string; components: any[] }) {
    const c = await
this.db.company(u.companyId); if
(!b.components?.length) b.components =
SAMPLE_STRUCT;
    computePay(c, u, b.components, 100000,
30, 30, ym(), []); // validates every
formula/reference
    const s = await
this.db.addRec('STRUCT', { companyId:
u.companyId, status: 'ACTIVE', data: {
name: b.name, components: b.components }
});
    return { structure: s, notified: (await
lawsFor(this.db, c)).map(l => l.data) };
}

@Put('structures/:id') @Roles('HR',
'HR_MANAGER', 'HRBP', 'ADMIN') async
upStruct(@Me() u: User, @Param('id') id:
string, @Body() b: { name?: string;
components: any[] }) { const r = await
this.db.rec('STRUCT', id); computePay(await

```

```

this.db.company(u.companyId), u,
b.components, 100000, 30, 30, ym(), []);
r.data = { ...r.data, ...b }; return
this.db.recs.save(r); }

/** Assign structure + monthly CTC. If
result breaches minimum wage, an HR/Admin
digital attestation is required. */
@Post('assign') @Roles('HR',
'HR_MANAGER', 'HRBP', 'ADMIN') async
assign(@Me() u: User, @Body() b: { userId:
string; structureId: string; ctc: number;
attestation?: { signature: string;
accepted: boolean } }) {
    const c = await
this.db.company(u.companyId), st = await
this.db.rec('STRUCT', b.structureId), laws
= await lawsFor(this.db, c);
    const calc = computePay(c, await
this.db.user(b.userId), st.data.components,
b.ctc, 30, 30, ym(), laws), viol: string[]
= [];
    const mw = laws.find(l => l.data.kind
=== 'MIN_WAGE')?.data; if (mw && calc.gross
< mw.monthly) viol.push(`Gross ₹
${calc.gross} is below the minimum wage ₹
${mw.monthly} for ${c.state}${mw.sample ? '
(sample value)' : ''}`);
    if (viol.length) {
        if (!b.attestation?.accepted ||
!b.attestation.signature) throw new
ConflictException({ requiresAttestation:
true, violations: viol, message: 'Rule
violation. Re-submit with attestation
{signature, accepted:true} signed by HR /

```

```

Admin / Super Admin.' });
    await this.needSigner(u); await
this.db.addRec('ATTEST', { companyId:
u.companyId, ownerId: u.id, refId:
b.userId, data: { violations: viol,
signature: b.attestation.signature,
signedAs: u.role, statement: 'I acknowledge
the applicable rule and knowingly configure
this salary despite the violation.', at:
now(), ctc: b.ctc } });
    }
    const ex = (await
this.db.listRec('EMPSAL', { ownerId:
b.userId }))[0];
    if (ex) { ex.refId = b.structureId;
ex.data = { ctc: b.ctc }; return
this.db.recs.save(ex); }
    return this.db.addRec('EMPSAL', {
companyId: u.companyId, ownerId: b.userId,
refId: b.structureId, data: { ctc: b.ctc }
});
    }
    @Get('assignments') @Roles('HR',
'HR_MANAGER', 'HRBP', 'FINANCE', 'ADMIN')
assignments(@Me() u: User) { return
this.db.listRec('EMPSAL', { companyId:
u.companyId }); }
    @Get('attestations') @Roles('HR_MANAGER',
'ADMIN') att(@Me() u: User) { return
this.db.listRec('ATTEST', { companyId:
u.companyId }); }
    /** Payroll-only clients upload
attendance: rows [{empId, paidDays}] */
    @Post('attendance/upload') @Roles('HR',

```

```

'HR_MANAGER', 'HRBP', 'ADMIN') async
upAtt(@Me() u: User, @Body() b: { month:
string; rows: { empId: string; paidDays:
number }[] }) {
    const old = (await
this.db.listRec('PAYATT', { companyId:
u.companyId, key: b.month }))[0];
    if (old) { old.data = { rows: b.rows };
return this.db.recs.save(old); } return
this.db.addRec('PAYATT', { companyId:
u.companyId, key: b.month, data: { rows:
b.rows } });
}
@Post('runs') @Roles('HR', 'HR_MANAGER',
'HRBP', 'ADMIN') async run(@Me() u: User,
@Body() b: { month: string }) {
    const month = b.month || ym(), c =
await this.db.company(u.companyId), laws =
await lawsFor(this.db, c), days =
daysInMonth(month);
    if ((await this.db.listRec('PAYRUN', {
companyId: u.companyId, key: month
})).find(r => r.status !== 'DRAFT')) throw
new ConflictException('Payroll for this
month is already locked/paid');
    const upl = (await
this.db.listRec('PAYATT', { companyId:
u.companyId, key: month }))[0]?.data.rows
as any[] | undefined;
    const lines: any[] = [];
    for (const es of await
this.db.listRec('EMPSAL', { companyId:
u.companyId })) {
        const e = await

```

```

this.db.users.findOneBy({ id: es.ownerId
}); if (!e || !e.active) continue; const st
= await this.db.rec('STRUCT', es.refId);
    let paid = days; const up =
upl?.find(r => r.empId === e.empId);
    if (up) paid = Math.min(days,
up.paidDays); else { const lop = await
this.db.att.count({ where: { userId: e.id,
date: Like(month + '%'), status:
In(['ABSENT', 'LOP']) } }); paid =
Math.max(0, days - lop); }
    const pay = computePay(c, e,
st.data.components, es.data.ctc, paid,
days, month, laws);
    lines.push({ userId: e.id, empId:
e.empId, name: e.name, uan: e.profile?.uan,
esiNo: e.profile?.esiNo, pan:
e.profile?.pan, doj: e.doj, bank: {
account: e.profile?.bankAccount, ifsc:
e.profile?.ifsc }, paidDays: paid, days,
...pay });
    }
    const totals = { employees:
lines.length, gross:
round2(lines.reduce((a, l) => a + l.gross,
0)), net: round2(lines.reduce((a, l) => a +
l.net, 0)), employerCost:
round2(lines.reduce((a, l) => a + l.gross +
l.statutory.pfER + l.statutory.esiER, 0))
};
    const ex = (await
this.db.listRec('PAYRUN', { companyId:
u.companyId, key: month }))[0];
    if (ex) { ex.data = { lines, totals };

```

```

return this.db.recs.save(ex); }

    return this.db.addRec('PAYRUN', {
companyId: u.companyId, key: month, status:
'DRAFT', data: { lines, totals } });
    }

    @Get('runs') @Roles('HR', 'HR_MANAGER',
'HRBP', 'FINANCE', 'ADMIN') async
runs(@Me() u: User) { return (await
this.db.listRec('PAYRUN', { companyId:
u.companyId })).map(r => ({ id: r.id,
month: r.key, status: r.status, totals:
r.data.totals })); }

    @Get('runs/:id') @Roles('HR',
'HR_MANAGER', 'HRBP', 'FINANCE', 'ADMIN')
runOne(@Param('id') id: string) { return
this.db.rec('PAYRUN', id); }

    @Post('runs/:id/lock')
@Roles('HR_MANAGER', 'ADMIN') async
lock(@Param('id') id: string) { const r =
await this.db.rec('PAYRUN', id); r.status =
'LOCKED'; await this.db.recs.save(r); await
this.n.send(r.data.lines.map((l: any) =>
l.userId), { title: `Payslip for ${r.key}
is available`, type: 'PAYROLL', link:
`/payroll/payslip?month=${r.key}` });
return r; }

    @Post('runs/:id/pay') @Roles('FINANCE',
'ADMIN') async pay(@Param('id') id: string)
{ const r = await this.db.rec('PAYRUN',
id); if (r.status !== 'LOCKED') throw new
BadRequestException('Lock the run first');
r.status = 'PAID'; return
this.db.recs.save(r); }

    @Get('payslip') async payslip(@Me() u:

```

```

User, @Query('month') month = ym()) {
    const r = (await
this.db.listRec('PAYRUN', { companyId:
u.companyId, key: month })).find(x =>
x.status !== 'DRAFT'); const l =
r?.data.lines.find((x: any) => x.userId ===
u.id);
    if (!l) throw new NotFoundException('No
payslip for this month'); return { month,
company: (await
this.db.company(u.companyId)).name, ...l };
}
/** Government-format downloads: pf (ECR
text), esi, pt, lwf, bonus, gratuity, bank
(NEFT) */
@Get('runs/:id/report/:kind')
@Roles('HR', 'HR_MANAGER', 'HRBP',
'FINANCE', 'ADMIN') async report(@Me() u:
User, @Param('id') id: string,
@param('kind') kind: string, @Res() res:
any) {
    const r = await this.db.rec('PAYRUN',
id), L: any[] = r.data.lines, c = await
this.db.company(u.companyId), s = cfg(c),
laws = await lawsFor(this.db, c); let out =
'', ext = 'csv';
    if (kind === 'pf') { ext = 'txt'; out =
L.filter(l => l.statutory.pfEE).map(l =>
[l.uan || '', l.name, Math.round(l.gross),
l.statutory.epfWages, l.statutory.epsWages,
l.statutory.epsWages, l.statutory.pfEE,
l.statutory.epsER, l.statutory.epfER,
l.days - l.paidDays,
0].join('#~#')).join('\n'); }

```

```

    else if (kind === 'esi') out =
    csv([[ 'IP Number', 'IP Name', 'No of Days',
    'Total Monthly Wages', 'Reason Code for
    Zero workings days', 'Last Working Day'],
    ...L.filter(l => l.statutory.esiEE).map(l
    => [l.esiNo || '', l.name, l.paidDays,
    l.gross, l.paidDays ? '' : '1', '']));
    else if (kind === 'pt') out =
    csv([[ 'Emp ID', 'Name', 'Gross',
    'Professional Tax', 'State'], ...L.filter(l
    => l.statutory.pt).map(l => [l.empId,
    l.name, l.gross, l.statutory.pt,
    c.state])]);
    else if (kind === 'lwf') out =
    csv([[ 'Emp ID', 'Name', 'Employee share',
    'Employer share'], ...L.filter(l =>
    l.statutory.lwf).map(l => [l.empId, l.name,
    l.statutory.lwf, l.statutory.lwfER])]);
    else if (kind === 'bonus') { const b =
    laws.find(x => x.data.kind ===
    'BONUS')?.data || { pct: 8.33, calcCeiling:
    7000, wageCeiling: 21000 }; out =
    csv([[ 'Emp ID', 'Name', 'Bonus wage',
    'Monthly accrual', 'Annual (approx)'],
    ...L.filter(l => l.gross <=
    b.wageCeiling).map(l => { const w =
    Math.min(l.basic, Math.max(b.calcCeiling,
    laws.find(x => x.data.kind ===
    'MIN_WAGE')?.data.monthly || 0)); return
    [l.empId, l.name, w, round2((w * b.pct) /
    100), round2((w * b.pct * 12) / 100)]};
    }]); }
    else if (kind === 'gratuity') { const g
    = laws.find(x => x.data.kind ===

```

```

'GRATUITY')?.data || { days: 15, perDays:
26, minYears: 5 }; out = csv(['Emp ID',
'Name', 'DOJ', 'Years', 'Eligible',
'Monthly provision', 'Accrued liability'],
...L.map(l => { const y =
round2(diffDays(l.doj || ymd(), ymd()) /
365.25), per = (l.basic * g.days) /
g.perDays; return [l.empId, l.name, l.doj,
y, y >= g.minYears ? 'YES' : 'NO',
round2(per / 12), round2(per *
Math.floor(y))]; })); }
    else if (kind === 'bank') out =
csv(['Beneficiary Name', 'Account No',
'IFSC', 'Amount', 'Narration', 'Emp ID'],
...L.map(l => [l.name, l.bank.account,
l.bank.ifsc, l.net, `Salary ${r.key}`,
l.empId]));
    else throw new
BadRequestException('kind must be
pf|esi|pt|lwf|bonus|gratuity|bank');
    res.setHeader('Content-Type',
'text/plain'); res.setHeader('Content-
Disposition', `attachment;
filename="${kind}_${r.key}.${ext}"`);
res.send(out);
}
}

/* ===== $7 LIVE TRACKING
+ TRAVEL CLAIMS (Feature: tracking)
===== */
@Ctl('tracking') @Feat('tracking') class
TrackingCtl {
    constructor(private db: Db, private n:

```

```

Notifier, private ap: Approvals) {}

@Post('start') async start(@Me() u: User,
@Body() b: { purpose?: string }) {
    if ((await this.db.listRec('TRIP', {
ownerId: u.id, status: 'ACTIVE' })).length)
throw new ConflictException('Trip already
active');
    return this.db.addRec('TRIP', {
companyId: u.companyId, ownerId: u.id,
status: 'ACTIVE', data: { purpose:
b?.purpose, startedAt: now(), distanceM: 0,
lastPingAt: now(), lastMoveAt: now(),
enabled: true, path: [], idle: [],
offSince: null, rmNotified: false,
hrNotified: false } });
}

/** Mobile app sends a ping every ~30 s
(also with enabled:false when the user
switches location off). */
@Post('ping') async ping(@Me() u: User,
@Body() b: { lat?: number; lng?: number;
enabled?: boolean }) {
    const t = (await
this.db.listRec('TRIP', { ownerId: u.id,
status: 'ACTIVE' }))[0]; if (!t) throw new
BadRequestException('No active trip');
    const d = t.data, c = await
this.db.company(u.companyId), s = cfg(c);
d.lastPingAt = now();
    if (b.enabled === false) { d.enabled =
false; if (!d.offSince) { d.offSince =
now(); } if (!d.rmNotified) { d.rmNotified
= true; await this.n.send(u.rmId || (await
this.db.ids(u.companyId, HRS)), { title:

```

```

`${u.name} turned location OFF`, body:
'Live tracking is interrupted', type:
'TRACKING', sms: true, link:
'/tracking/live' }); } }
    else {
        d.enabled = true; d.offSince = null;
d.rmNotified = false; d.hrNotified = false;
        if (d.lastLat !== undefined) { const
m = hav(d.lastLat, d.lastLng, b.lat,
b.lng); if (m >= 30) { d.distanceM += m;
d.lastMoveAt = now(); d.path.push([b.lat,
b.lng, now()]); if (d.path.length > 1500)
d.path.shift(); if (d.idle.length &&
!d.idle[d.idle.length - 1].to)
d.idle[d.idle.length - 1].to = now(); }
        else if ((Date.now() - +new
Date(d.lastMoveAt)) / 6e4 >= s.idleMinutes
&& !(d.idle.length && !d.idle[d.idle.length
- 1].to)) d.idle.push({ from: d.lastMoveAt,
lat: b.lat, lng: b.lng, to: null }); }
        else d.path.push([b.lat, b.lng,
now()]);
        d.lastLat = b.lat; d.lastLng = b.lng;
    }
    t.data = { ...d }; await
this.db.recs.save(t); return { distanceKm:
round2(d.distanceM / 1000), idleEvents:
d.idle.length };
}
/** Ending the trip auto-generates the
travel claim: distance × fixed rate per km
(+ optional bills). */
@Post('stop') async stop(@Me() u: User,
@Body() b: { bills?: { name: string;

```

```

amount: number; url?: string }[] }) {
    const t = (await
this.db.listRec('TRIP', { ownerId: u.id,
status: 'ACTIVE' }))[0]; if (!t) throw new
BadRequestException('No active trip');
    t.status = 'ENDED'; t.data = {
...t.data, endedAt: now() }; await
this.db.recs.save(t);
    const km = round2(t.data.distanceM /
1000); if (km <= 0 && !b?.bills?.length)
return { trip: t, claim: null };
    return { trip: t, claim: await
createClaim(this.db, this.ap, this.n, u, {
tripId: t.id, distanceKm: km, bills:
b?.bills || [] }) };
}
@Get('trips') async trips(@Me() u: User)
{ return this.db.listRec('TRIP', { ownerId:
u.id }); }
@Get('live') @Roles('RM', 'HR', 'HRBP',
'HR_MANAGER', 'ADMIN') async live(@Me() u:
User) {
    const ids = HRS.includes(u.role) ||
ADMINS.includes(u.role) ? null : (await
this.db.teamOf(u.id)).map(x => x.id);
    const act = (await
this.db.listRec('TRIP', { companyId:
u.companyId, status: 'ACTIVE' })).filter(t
=> !ids || ids.includes(t.ownerId));
    return Promise.all(act.map(async t =>
({ userId: t.ownerId, name: (await
this.db.user(t.ownerId)).name, lat:
t.data.lastLat, lng: t.data.lastLng, map:
t.data.lastLat ? mapUrl(t.data.lastLat,

```

```

t.data.lastLng) : null, distanceKm:
round2(t.data.distanceM / 1000), idle:
t.data.idle.filter((i: any) =>
!i.to).length > 0, locationOffSince:
t.data.offSince, lastPingAt:
t.data.lastPingAt }));
    }
  }
  async function createClaim(db: Db, ap:
Approvals, n: Notifier, u: User, o: {
tripId?: string; distanceKm?: number;
bills?: any[]; title?: string }) {
    const c = await db.company(u.companyId),
s = cfg(c), travel = round2((o.distanceKm
|| 0) * s.ratePerKm), reimb =
round2((o.bills || []).reduce((a, b) => a +
Number(b.amount || 0), 0)), total =
round2(travel + reimb), over = total >
s.claimLimit;
    const a = await ap.create('CLAIM', u, {
tripId: o.tripId, distanceKm: o.distanceKm
|| 0, ratePerKm: s.ratePerKm, travelAmount:
travel, bills: o.bills || [],
reimbursementAmount: reimb, total, limit:
s.claimLimit, overLimit: over, paid: false
}, ['RM', 'HR', 'FINANCE'], `${o.title ||
'Travel claim'} ₹${total} - ${u.name}`);
    if (over) await n.send([u.rmId, ...(await
db.ids(u.companyId, HRS))].filter(Boolean),
{ title: `Claim above limit: ${u.name} ₹
${total} (limit ₹${s.claimLimit})`, type:
'CLAIM', link: `/approvals/${a.id}` }]);
    return a;
  }
}

```

```

@Ctl('claims') @Feat('tracking') class
ClaimsCtl {
  constructor(private db: Db, private ap:
Approvals, private n: Notifier) {}
  @Post() async create(@Me() u: User,
@Body() b: { title?: string; bills: { name:
string; amount: number; url?: string }[] })
{ if (!b.bills?.length) throw new
BadRequestException('Attach at least one
bill'); return createClaim(this.db,
this.ap, this.n, u, { bills: b.bills,
title: b.title || 'Reimbursement' }); }
  @Get('mine') async mine(@Me() u: User) {
return Promise.all((await
this.db.approvals.find({ where: {
requesterId: u.id, type: 'CLAIM' }, order:
{ createdAt: 'DESC' } })).map(a =>
this.ap.view(a))); }
  @Post('/:id/bills') async addBills(@Me()
u: User, @Param('id') id: string, @Body()
b: { bills: any[] }) {
    const a = await
this.db.approvals.findOneByOrFail({ id,
requesterId: u.id, type: 'CLAIM' }); if
(a.status !== 'PENDING' || a.step > 0)
throw new BadRequestException('Bills can
only be added before the RM acts');
    const bills = [...a.payload.bills,
...b.bills], reimb =
round2(bills.reduce((x: number, y: any) =>
x + Number(y.amount), 0)); a.payload = {
...a.payload, bills, reimbursementAmount:
reimb, total: round2(a.payload.travelAmount
+ reimb), overLimit: a.payload.travelAmount

```

```

+ reimb > a.payload.limit }; return
this.db.approvals.save(a);
}
@Get('rules') async rules(@Me() u: User)
{ const s = cfg(await
this.db.company(u.companyId)); return {
ratePerKm: s.ratePerKm, claimLimit:
s.claimLimit }; }
@Put('rules') @Roles('HR', 'HR_MANAGER',
'ADMIN') async setRules(@Me() u: User,
@Body() b: { ratePerKm?: number;
claimLimit?: number }) {
const c = await
this.db.company(u.companyId); c.settings =
{ ...cfg(c), ...(b.ratePerKm !== undefined
&& { ratePerKm: b.ratePerKm }), ...
(b.claimLimit !== undefined && {
claimLimit: b.claimLimit }) }; await
this.db.companies.save(c);
await this.n.send(await
this.db.ids(u.companyId, ['EMPLOYEE', 'RM',
...HRS, 'FINANCE', 'DEPT_HEAD']), { title:
'Travel claim rules updated', body: `₹
${cfg(c).ratePerKm}/km · limit ₹
${cfg(c).claimLimit} per claim`, type:
'CLAIM' }); return { ratePerKm:
cfg(c).ratePerKm, claimLimit:
cfg(c).claimLimit };
}
@Post('/:id/pay') @Roles('FINANCE') async
pay(@Me() u: User, @Param('id') id: string)
{ const a = await
this.db.approvals.findOneByOrFail({ id,
companyId: u.companyId, type: 'CLAIM' });

```

```

if (a.status !== 'APPROVED') throw new
BadRequestException('Only fully approved
claims can be paid'); a.payload = {
...a.payload, paid: true, paidAt: now() };
await this.db.approvals.save(a); await
this.n.send(a.requesterId, { title: `Claim
paid: ₹${a.payload.total}`, type: 'CLAIM',
sms: true }); return a; }

@Get('finance-queue') @Roles('FINANCE')
async queue(@Me() u: User) { return (await
this.db.approvals.find({ where: {
companyId: u.companyId, type: 'CLAIM',
status: 'APPROVED' } })).filter(a =>
!a.payload.paid); }
}

/* ===== §8 PERFORMANCE:
KPI / KRI + PIP (Feature: performance)
===== */

const prevMonth = (m: string, k = 1) =>
ym(addMonths(m + '-15', -k));
@Ctl('performance') @Feat('performance')
class PerformanceCtl {
  constructor(private db: Db, private n:
Notifier, private ap: Approvals) {}
  @Post('kpi') @Roles('RM', 'HR', 'HRBP',
'HR_MANAGER') async setKpi(@Me() u: User,
@Body() b: { userId: string; month?:
string; items?: { name: string; target:
number; unit?: string }[]; sameAsPrevious?:
boolean }) {
    const e = await this.db.user(b.userId),
    month = b.month || ym(); if (e.rmId !==
u.id && !HRS.includes(u.role)) throw new

```

```

ForbiddenException('Not your team member');
    if ((await this.db.listRec('KPI', {
ownerId: e.id, key: month }))[0]) throw new
ConflictException('KPI already set for this
month');
    let items = b.items;
    if (b.sameAsPrevious) { const prev =
(await this.db.listRec('KPI', { ownerId:
e.id })).filter(k => k.key <
month).sort((a, c) => (a.key < c.key ? 1 :
-1))[0]; if (!prev) throw new
BadRequestException('No previous month
KPI'); items = prev.data.items.map((i: any)
=> ({ name: i.name, target: i.target, unit:
i.unit })); }
    if (!items?.length) throw new
BadRequestException('items required');
    const k = await this.db.addRec('KPI', {
companyId: e.companyId, ownerId: e.id, key:
month, status: 'SET', data: { items, setBy:
u.id, sameAsPrevious: !!b.sameAsPrevious }
});
    await this.n.send(e.id, { title: `KPIs
set for ${month}`, type: 'KPI', link:
`/performance/kpi/${k.id}` }); return k;
}
@Get('kpi/mine') mine(@Me() u: User) {
return this.db.listRec('KPI', { ownerId:
u.id }); }
@Get('kpi/team') @Roles('RM', 'HR',
'HRBP', 'HR_MANAGER') async team(@Me() u:
User, @Query('month') m = ym()) { const ids
= HRS.includes(u.role) ? null : (await
this.db.teamOf(u.id)).map(x => x.id);

```

```

return (await this.db.listRec('KPI', {
  companyId: u.companyId, key: m })).filter(k
=> !ids || ids.includes(k.ownerId)); }

  @Post('kpi/:id/achieve') async
  achieve(@Me() u: User, @Param('id') id:
  string, @Body() b: { items: { name: string;
  achieved: number }[] }) {
    const k = await this.db.rec('KPI', id);
    if (k.ownerId !== u.id) throw new
    ForbiddenException();
    k.data.items = k.data.items.map((i:
    any) => ({ ...i, achieved: b.items.find(x
    => x.name === i.name)?.achieved ??
    i.achieved })); k.status = 'SUBMITTED';
    k.data = { ...k.data, scorePct:
    score(k.data.items) }; await
    this.db.recs.save(k);
    await this.n.send(u.rmId, { title:
    `${u.name} submitted ${k.key}
    achievements`, type: 'KPI', link:
    `/performance/kpi/${id}` }); return k;
  }

  @Post('kpi/:id/verify') @Roles('RM',
  'HR', 'HRBP', 'HR_MANAGER') async
  verify(@Me() u: User, @Param('id') id:
  string, @Body() b: { approve: boolean;
  remarks?: string }) {
    const k = await this.db.rec('KPI', id),
    e = await this.db.user(k.ownerId); if
    (e.rmId !== u.id && !HRS.includes(u.role))
    throw new ForbiddenException();
    k.status = b.approve ? 'APPROVED' :
    'REJECTED'; k.data = { ...k.data, remarks:
    b.remarks, verifiedBy: u.id }; await

```

```

this.db.recs.save(k);
    if (b.approve) { // 3 consecutive
months below 50% → alert HR to trigger PIP
        const last3 = await Promise.all([0,
1, 2].map(i => this.db.listRec('KPI', {
ownerId: e.id, key: prevMonth(k.key, i),
status: 'APPROVED' })).then(r => r[0]));
        if (last3.every(x => x &&
x.data.scorePct < 50)) await
this.n.send(await this.db.ids(e.companyId,
HRS), { title: `PIP alert: ${e.name} below
50% for 3 consecutive months`, body:
`Scores: ${last3.map(x => x.data.scorePct +
'%').join(', ')}`, type: 'PIP', dedupe:
`pipalert:${e.id}:${k.key}`, link:
`/performance/pip/new?userId=${e.id}`, sms:
true });
    }
    await this.n.send(e.id, { title: `KPI
${k.key} ${k.status}`, body: b.remarks,
type: 'KPI' }); return k;
}
@Post('pip') @Roles('HR', 'HRBP',
'HR_MANAGER') async pip(@Me() u: User,
@Body() b: { userId: string; endDate:
string; terms: string; meetingAt?: string;
mode?: 'ONLINE' | 'CALENDAR' }) {
    const e = await this.db.user(b.userId),
start = ymd();
    const p = await this.db.addRec('PIP', {
companyId: e.companyId, ownerId: e.id,
status: 'PENDING_ACK', data: { startDate:
start, endDate: b.endDate, terms: b.terms,
hrId: u.id, rmId: e.rmId } });

```

```

        if (b.meetingAt) { const m = await
this.ap.meets(e.companyId, u.id, 'PIP',
`PIP kick-off - ${e.name}`, b.meetingAt,
30, [e.id, e.rmId].filter(Boolean), {
pipId: p.id, mode: b.mode || 'ONLINE' });
p.data = { ...p.data, meetingId: m.id };
await this.db.recs.save(p); }
        await this.n.send([e.id,
e.rmId].filter(Boolean), { title: `PIP
initiated for ${e.name}`, body: `Ends
${b.endDate}. Employee must acknowledge
with e-signature.`, type: 'PIP', link:
`/performance/pip/${p.id}`, sms: true });
return p;
    }
    @Get('pip') async pips(@Me() u: User) {
return this.db.listRec('PIP',
HRS.includes(u.role) ? { companyId:
u.companyId } : u.role === 'RM' ? {
companyId: u.companyId } : { ownerId: u.id
}); }
    @Post('pip/:id/ack') async ack(@Me() u:
User, @Param('id') id: string, @Body() b: {
signature: string }) { const p = await
this.db.rec('PIP', id); if (p.ownerId !==
u.id) throw new ForbiddenException(); if
(!b.signature) throw new
BadRequestException('e-signature
required'); p.status = 'ACTIVE'; p.data = {
...p.data, signature: b.signature, ackAt:
now() }; return this.db.recs.save(p); }
    @Post('pip/:id/outcome') @Roles('HR',
'HRBP', 'HR_MANAGER') async outcome(@Me()
u: User, @Param('id') id: string, @Body()

```

```

b: { result: 'SUCCESS' | 'EXTEND' |
'SEPARATE'; newEnd?: string; remarks?:
string }) {
    const p = await this.db.rec('PIP', id),
    e = await this.db.user(p.ownerId);
    if (b.result === 'SUCCESS') p.status =
'COMPLETED_SUCCESS'; else if (b.result ===
'EXTEND') { p.data = { ...p.data, endDate:
b.newEnd, extended: true }; p.status =
'ACTIVE'; } else { p.status =
'COMPLETED_FAILED'; await
this.ap.create('TERMINATION', e, { reason:
'PIP not cleared' }, ['HR_MANAGER'],
`Separation (PIP failed): ${e.name}`); }
    p.data = { ...p.data, outcome:
b.result, remarks: b.remarks }; await
this.db.recs.save(p); await
this.n.send([e.id, e.rmId].filter(Boolean),
{ title: `PIP outcome: ${b.result}`, type:
'PIP' }); return p;
    }
}

const score = (items: any[]) =>
Math.round(items.reduce((s, i) => s +
Math.min(150, ((i.achieved ?? 0) /
(i.target || 1)) * 100), 0) /
items.length);

/* ===== §9 ENGAGEMENT +
FEED (Feature: engagement)
===== */

const ACTIVITIES = [
    { type: 'GAME', title: 'Memory Match (2
min)', url: '/games/memory' }, { type:

```

```

'STRETCH', title: 'Desk stretch: roll
shoulders ×10, neck tilt ×5 each side' },
  { type: 'QUIZ', title: 'Quick quiz: Which
planet has the most moons?', answer:
'Saturn' }, { type: 'BREATHE', title: '4-7-
8 breathing, 4 rounds' }, { type:
'GRATITUDE', title: 'Send a thank-you to a
teammate on the feed' },
];
@Ctl('engagement') @Feat('engagement')
class EngagementCtl {
  constructor(private db: Db, private n:
Notifier, private ap: Approvals) {}
  @Post('recognition') @Roles('RM',
'DEPT_HEAD', 'HR', 'HRBP', 'HR_MANAGER')
  async recognise(@Me() u: User, @Body() b: {
    userId: string; award: string; month?:
    string; reason?: string; monetary?:
    boolean; amount?: number }) {
    const e = await this.db.user(b.userId),
    month = b.month || ym();
    const r = await
    this.db.addRec('RECOGNITION', { companyId:
    u.companyId, ownerId: e.id, key: month,
    status: b.monetary ? 'PENDING_APPROVAL' :
    'GIVEN', data: { ...b, month, by: u.name }
    });
    await this.db.addRec('POST', {
    companyId: u.companyId, ownerId: u.id,
    status: 'PUBLISHED', data: { kind: 'AWARD',
    text: `🏆 ${b.award} – ${e.name}
    (${b.month}). ${b.reason || ''}`, awardee:
    e.id, likes: [] } });
    await this.n.send(e.id, { title: `You

```

```

received "${b.award}"!`, body: b.reason,
type: 'AWARD', sms: true });
    await this.n.send(await
this.db.ids(u.companyId, HRS), { title:
`Recognition: ${e.name} – ${b.award}`,
body: b.monetary ? `Monetary ₹${b.amount} –
approval needed` : 'Non-monetary', type:
'AWARD' });
    if (b.monetary) { const a = await
this.ap.create('REWARD', u, {
recognitionId: r.id, awardee: e.id, amount:
b.amount }, ['HR', 'FINANCE'], `Reward ₹
${b.amount} for ${e.name}`); r.refId =
a.id; await this.db.recs.save(r); }
    return r;
  }
  @Get('awards') awards(@Me() u: User,
@Query('month') m?: string) { return
this.db.listRec('RECOGNITION', { companyId:
u.companyId, ...(m ? { key: m } : {}) }); }
  @Post('water') async water(@Me() u: User,
@Body() b: { status: 'DRANK' | 'LATER' }) {
    if (b.status === 'LATER') { await
this.db.addRec('SNOOZE', { ownerId: u.id,
status: 'OPEN', data: { dueAt: new
Date(Date.now() + 20 * 6e4).toISOString() }
}); return { snoozedMinutes: 20 }; }
    await this.db.addRec('WATER', {
companyId: u.companyId, ownerId: u.id, key:
ymd(), data: {} }); return { today: (await
this.db.listRec('WATER', { ownerId: u.id,
key: ymd() })).length };
  }
  /** App reports phone idle minutes; ≥60

```

```

returns a small game/activity. */
  @Post('idle') async idle(@Me() u: User,
  @Body() b: { minutes: number }) { if
  (b.minutes < 60) return { activity: null };
  const a =
  ACTIVITIES[Math.floor(Math.random() *
  ACTIVITIES.length)]; await
  this.db.addRec('ACTIVITY', { ownerId: u.id,
  key: ymd(), data: a }); return { activity:
  a }; }

  @Post('one-on-ones/schedule')
  @Roles('HRBP', 'HR_MANAGER', 'HR') async
  sched(@Me() u: User) { return
  scheduleOneOnOnes(this.db, this.ap,
  u.companyId); }

  @Get('one-on-ones') async oneOnes(@Me()
  u: User) { return (await
  this.db.meetings.find({ where: { companyId:
  u.companyId, kind: 'ONE_ON_ONE' }, order: {
  startsAt: 'DESC' } })).filter(m =>
  m.attendeeIds.includes(u.id)); }

  @Post('one-on-ones/:id/feedback')
  @Roles('HRBP', 'HR_MANAGER', 'HR') async
  oneFb(@Me() u: User, @Param('id') id:
  string, @Body() b: { mood?: string; notes:
  string; actionItems?: string[] }) {
    const m = await
    this.db.meetings.findOneByOrFail({ id,
    kind: 'ONE_ON_ONE' }); m.status = 'DONE';
    await this.db.meetings.save(m);
    return this.db.addRec('ONE_ON_ONE_FB',
    { companyId: u.companyId, ownerId: u.id,
    refId: id, data: { ...b, employeeId:
    m.ctx?.employeeId } });

```

```

    }
    @Get('one-on-ones/feedback')
    @Roles('HRBP', 'HR_MANAGER', 'HR')
    ofb(@Me() u: User) { return
    this.db.listRec('ONE_ON_ONE_FB',
    HRS.includes(u.role) && u.role !== 'HRBP' ?
    { companyId: u.companyId } : { ownerId:
    u.id }); }
    }
    /** Default monthly 5-min 1:1 with each
    employee's HRBP, back-to-back 5-minute
    slots from 11:00 on the 5th. */
    async function scheduleOneOnOnes(db: Db,
    ap: Approvals, companyId: string) {
        const emps = (await
    db.employees(companyId)).filter(e =>
    e.hrbpId, by: Record<string, User[]> = {});
    emps.forEach(e => (by[e.hrbpId] =
    by[e.hrbpId] || []).push(e)); let n = 0;
        const day = new Date(); day.setDate(5);
    if (day < new Date())
    day.setMonth(day.getMonth() + (new
    Date().getDate() > 5 ? 1 : 0));
        for (const [hrbp, list] of
    Object.entries(by)) for (let i = 0; i <
    list.length; i++) {
            const at = new Date(day);
    at.setHours(11, i * 5, 0, 0);
            const exists = (await
    db.meetings.find({ where: { companyId,
    kind: 'ONE_ON_ONE', hostId: hrbp }
    })).some(m => m.ctx?.employeeId ===
    list[i].id && m.ctx?.month === ym(at));
            if (!exists) { await

```

```

ap.meets(companyId, hrbp, 'ONE_ON_ONE',
`Monthly 1:1 - ${list[i].name}`,
at.toISOString(), 5, [list[i].id], {
  employeeId: list[i].id, month: ym(at) });
n++; }
}
return { scheduled: n };
}

@Ctl('feed') class FeedCtl {
  constructor(private db: Db) {}
  @Get() async feed(@Me() u: User) {
    const items: any[] = []; if (u.zodiac)
    items.push({ kind: 'HOROSCOPE', pinned:
true, at: now(), ...horoscope(u.zodiac) });
    if (u.companyId) {
      const posts = (await
this.db.listRec('POST', { companyId:
u.companyId })).slice(0, 50);
      items.push(...posts.map(p => ({ id: p.id,
kind: p.data.kind || 'POST', at:
p.createdAt, text: p.data.text, likes:
(p.data.likes || []).length, by: p.ownerId
})));
      const today = ymd().slice(5); for
(const e of await
this.db.employees(u.companyId)) { if
(e.dob?.slice(5) === today) items.push({
kind: 'BIRTHDAY', at: now(), text: `🎂
Happy Birthday ${e.name}!` }); if (e.doj &&
e.doj.slice(5) === today && e.doj.slice(0,
4) !== ymd().slice(0, 4)) items.push({
kind: 'ANNIVERSARY', at: now(), text: `🎉
${e.name} completes ${+ymd().slice(0, 4) -
+e.doj.slice(0, 4)} year(s) today!` }); }

```

```

    }
    return items.sort((a, b) => (b.pinned ?
1 : 0) - (a.pinned ? 1 : 0) || (a.at < b.at
? 1 : -1));
    }
    @Get('horoscope') hs(@Me() u: User) { if
(!u.zodiac) throw new
BadRequestException('No zodiac on
profile'); return horoscope(u.zodiac); }
    @Post('posts') async post(@Me() u: User,
@Body() b: { text: string; kind?: string })
{ if (!u.companyId) throw new
ForbiddenException(); const kind = b.kind
=== 'ANNOUNCEMENT' && [...HRS,
...ADMINS].includes(u.role) ?
'ANNOUNCEMENT' : 'POST'; return
this.db.addRec('POST', { companyId:
u.companyId, ownerId: u.id, status:
'PUBLISHED', data: { kind, text: b.text,
likes: [] } }); }
    @Post('posts/:id/like') async like(@Me()
u: User, @Param('id') id: string) { const p
= await this.db.rec('POST', id); const l =
new Set<string>(p.data.likes || []);
l.has(u.id) ? l.delete(u.id) : l.add(u.id);
p.data = { ...p.data, likes: [...l] };
await this.db.recs.save(p); return { likes:
l.size }; }
}

/* ===== $10 TRAINING &
DEVELOPMENT (Feature: training)
===== */
const courseSuggestions = (skill: string)

```

```

=> [
  { provider: 'Coursera (paid)', title:
    `${skill} – top-rated specialisation`, url:
    `https://www.coursera.org/search?
    query=${encodeURIComponent(skill)}`, paid:
    true },
  { provider: 'LinkedIn Learning (paid)',
    title: `${skill} learning paths`, url:
    `https://www.linkedin.com/learning/search?
    keywords=${encodeURIComponent(skill)}`,
    paid: true },
  { provider: 'NPTEL (free)', title:
    `${skill} – NPTEL certification courses`,
    url: `https://nptel.ac.in/courses?
    search=${encodeURIComponent(skill)}`, paid:
    false },
];
@Ctl('training') @Feat('training') class
TrainingCtl {
  constructor(private db: Db, private n:
  Notifier) {}
  /** RM analyses the team every 2 months
  and records skill gaps; one paid + one free
  course is suggested per gap. */
  @Post('skill-gap') @Roles('RM', 'HR',
  'HRBP', 'HR_MANAGER') async gap(@Me() u:
  User, @Body() b: { userId: string; gaps:
  string[] }) {
    const e = await this.db.user(b.userId);
    if (e.rmId !== u.id &&
    !HRS.includes(u.role)) throw new
    ForbiddenException();
    const sugg = b.gaps.map(g => ({ skill:
    g, courses: courseSuggestions(g) }));

```

```

        const r = await
this.db.addRec('SKILLGAP', { companyId:
e.companyId, ownerId: e.id, key: ym(),
data: { gaps: b.gaps, suggestions: sugg,
by: u.id } });
        await this.n.send(e.id, { title:
'Skill-gap review shared – pick a course',
body: b.gaps.join(', '), type: 'TRAINING',
link: `/training/skill-gap/${r.id}` });
return r;
    }
    @Get('skill-gap') sg(@Me() u: User) {
return this.db.listRec('SKILLGAP',
HRS.includes(u.role) ? { companyId:
u.companyId } : { ownerId: u.id }); }
    @Post('enroll') async enroll(@Me() u:
User, @Body() b: { title: string; provider:
string; url?: string; source?: 'SUGGESTED'
| 'SELF' }) { return
this.db.addRec('TRAINING', { companyId:
u.companyId, ownerId: u.id, status:
'IN_PROGRESS', data: { ...b, source:
b.source || 'SELF', startedAt: now() } });
    }
    @Post('/:id/certificate') async cert(@Me()
u: User, @Param('id') id: string, @Body()
b: { certificateUrl: string; testScore?:
number }) {
        const t = await this.db.rec('TRAINING',
id); if (t.ownerId !== u.id) throw new
ForbiddenException(); t.status =
'COMPLETED'; t.data = { ...t.data, ...b,
completedAt: now() }; await
this.db.recs.save(t);

```

```

        await this.n.send([u.rmId, ...(await
this.db.ids(u.companyId,
HRS))].filter(Boolean), { title: `${u.name}
completed "${t.data.title}"`, type:
'TRAINING' }); return t;
    }
    @Get('mine') mine(@Me() u: User) { return
this.db.listRec('TRAINING', { ownerId: u.id
}); }
    @Get('all') @Roles('HR', 'HRBP',
'HR_MANAGER', 'RM') all(@Me() u: User) {
return this.db.listRec('TRAINING', {
companyId: u.companyId }); }
    /** Training calendar (POSH etc.). HR is
auto-notified 7 days before by cron. */
    @Post('calendar') @Roles('HR', 'HRBP',
'HR_MANAGER') async cal(@Me() u: User,
@Body() b: { title: string; date: string;
mandatory?: boolean; audience?: string }) {
const r = await
this.db.addRec('TRAINING_CAL', { companyId:
u.companyId, key: b.date, status:
'SCHEDULED', data: b }); await
this.n.send(await this.db.ids(u.companyId,
['EMPLOYEE', 'RM']), { title: `Training
scheduled: ${b.title}`, body: b.date, type:
'TRAINING' }); return r; }
    @Get('calendar') calendar(@Me() u: User)
{ return this.db.listRec('TRAINING_CAL', {
companyId: u.companyId }); }
    /** Anonymous suggestion to another team
→ becomes that department's brainstorm
topic. */
    @Post('suggestions') async suggest(@Me()

```

```

u: User, @Body() b: { team: string; text:
string }) { return this.db.addRec('IDEA', {
companyId: u.companyId, key: b.team,
status: 'OPEN', data: { text: b.text,
anonymous: true } }); }

@Get('suggestions') @Roles('RM',
'DEPT_HEAD', 'HR', 'HRBP', 'HR_MANAGER')
async sug(@Me() u: User, @Query('team')
team?: string) { return (await
this.db.listRec('IDEA', { companyId:
u.companyId, ...(team ? { key: team } : {}))
)).map(r => ({ id: r.id, team: r.key,
text: r.data.text, at: r.createdAt })); }

@Post('brainstorm') @Roles('RM',
'DEPT_HEAD') async bs(@Me() u: User,
@Body() b: { topic: string }) { const r =
await this.db.addRec('BRAINSTORM', {
companyId: u.companyId, ownerId: u.id,
status: 'OPEN', data: { topic: b.topic,
ideas: [] } }); await this.n.send((await
this.db.teamOf(u.id)).map(x => x.id), {
title: `Brainstorm: ${b.topic}`, body:
'Share your ideas', type: 'TRAINING', link:
`/training/brainstorm/${r.id}` }); return
r; }

@Get('brainstorm') async bsList(@Me() u:
User) { return (await
this.db.listRec('BRAINSTORM', { companyId:
u.companyId })).filter(b => b.ownerId ===
u.id || u.rmId === b.ownerId ||
HRS.includes(u.role)); }

@Post('brainstorm/:id/ideas') async
idea(@Me() u: User, @Param('id') id:
string, @Body() b: { text: string }) {

```

```

const r = await this.db.rec('BRAINSTORM',
id); r.data = { ...r.data, ideas:
[...r.data.ideas, { by: u.name, text:
b.text, at: now() }] }; await
this.db.recs.save(r); await
this.n.send(r.ownerId, { title: `New idea
on "${r.data.topic}"`, type: 'TRAINING' });
return r; }
}

/* ===== §11 COMPLAINTS &
SUGGESTIONS (Feature: complaints)
===== */
@Ctl('complaints') @Feat('complaints')
class ComplaintsCtl {
  constructor(private db: Db, private n:
Notifier) {}
  private key(u: User) { return
crypto.createHmac('sha256',
ENV('JWT_SECRET', 'nerkanal-dev-
secret')).update(u.id).digest('hex'); }
  /** Anonymous: HR never sees the author.
The reporter key is a keyed hash; the
notify target is encrypted server-side
only. */
  @Post() async raise(@Me() u: User,
@Body() b: { subject: string; body: string;
category?: string; anonymous?: boolean }) {
    const code = 'CMP-' +
rid(6).toUpperCase(), anon = b.anonymous
!= false;
    const r = await
this.db.addRec('COMPLAINT', { companyId:
u.companyId, ownerId: anon ? null : u.id,

```

```

key: this.key(u), status: 'OPEN', data: {
  code, subject: b.subject, body: b.body,
  category: b.category, anonymous: anon,
  notifyEnc: enc(u.id), timeline: [{ at:
    now(), status: 'OPEN', note: 'Complaint
    received' }] } });
    await this.n.send(await
this.db.ids(u.companyId, HRS), { title:
`New complaint ${code}: ${b.subject}`,
type: 'COMPLAINT', link:
`/complaints/${r.id}` }); return { id:
r.id, trackingCode: code, status: 'OPEN' };
  }
  @Get('mine') async mine(@Me() u: User) {
return (await this.db.listRec('COMPLAINT',
{ companyId: u.companyId, key: this.key(u
})).map(strip); }
  @Get() @Roles('HR', 'HRBP', 'HR_MANAGER',
'ADMIN') async all(@Me() u: User,
@Query('status') s?: string) { return
(await this.db.listRec('COMPLAINT', {
companyId: u.companyId, ...(s ? { status: s
} : {}) })).map(strip); }
  @Post('/:id/update') @Roles('HR', 'HRBP',
'HR_MANAGER') async upd(@Me() u: User,
@param('id') id: string, @Body() b: {
status: string; note: string;
meetingMinutesUrl?: string }) {
    const r = await
this.db.rec('COMPLAINT', id); r.status =
b.status; r.data = { ...r.data, timeline:
[...r.data.timeline, { at: now(), status:
b.status, note: b.note, minutesUrl:
b.meetingMinutesUrl, by: 'HR' }] }; await

```

```

this.db.recs.save(r);
    await
this.n.send(dec(r.data.notifyEnc), { title:
`Complaint ${r.data.code}: ${b.status}`,
body: b.note, type: 'COMPLAINT' }); return
strip(r);
    }
}
const strip = (r: Rec) => { const {
notifyEnc, ...d } = r.data; return { id:
r.id, status: r.status, createdAt:
r.createdAt, ...d, ...(d.anonymous ? {} : {
raisedBy: r.ownerId }) }; };

/* ===== $12 EXIT
MANAGEMENT (Feature: exit)
===== */
@Ctl('exit') @Feat('exit') class ExitCtl {
    constructor(private db: Db, private ap:
Approvals, private n: Notifier) {}
    @Post('resign') async resign(@Me() u:
User, @Body() b: { reason: string; lwd?:
string }) { return
this.ap.create('RESIGNATION', u, b, ['RM'],
`Resignation - ${u.name}`); }
    @Get() @Roles('HR', 'HRBP', 'HR_MANAGER',
'RM', 'ADMIN') async list(@Me() u: User) {
return this.db.listRec('EXIT', { companyId:
u.companyId }); }
    @Get('mine') mine(@Me() u: User) { return
this.db.listRec('EXIT', { ownerId: u.id });
}
    /** HRBP sets notice period / last
working day. */

```

```

    @Post('/:id/notice') @Roles('HRBP', 'HR',
    'HR_MANAGER') async notice(@Me() u: User,
    @Param('id') id: string, @Body() b: {
    noticeDays?: number; lwd?: string }) {
        const x = await this.db.rec('EXIT',
    id), lwd = b.lwd || addDays(ymd(),
    b.noticeDays || 30); x.data = { ...x.data,
    lwd, noticeDays: b.noticeDays }; await
    this.db.recs.save(x);
        await this.n.send(x.ownerId, { title:
    `Your last working day is ${lwd}`, type:
    'EXIT', sms: true }); return x;
    }
    /** Clearance: Knowledge Transfer = RM ·
    Asset = HR · Access removal = Admin. */
    @Post('/:id/clear') async clear(@Me() u:
    User, @Param('id') id: string, @Body() b: {
    item: 'KT' | 'ASSET' | 'ACCESS'; note?:
    string }) {
        const x = await this.db.rec('EXIT',
    id), e = await this.db.user(x.ownerId),
    allowed = b.item === 'KT' ? e.rmId === u.id
    || u.role === 'SUPER_ADMIN' : b.item ===
    'ASSET' ? HRS.includes(u.role) :
    ADMINS.includes(u.role);
        if (!allowed) throw new
    ForbiddenException(`${b.item} clearance is
    done by ${b.item === 'KT' ? 'the reporting
    manager' : b.item === 'ASSET' ? 'HR' :
    'Admin'}`);
        x.data = { ...x.data, clearances: {
    ...x.data.clearances, [b.item]: { by:
    u.name, at: now(), note: b.note } } };
    await this.db.recs.save(x);

```

```

    return x;
  }
  @Post('/:id/close') @Roles('HR', 'HRBP',
    'HR_MANAGER') async close(@Me() u: User,
    @Param('id') id: string) {
    const x = await this.db.rec('EXIT',
    id), cl = x.data.clearances, e = await
    this.db.user(x.ownerId), c = await
    this.db.company(e.companyId);
    if (!cl.KT || !cl.ASSET || !cl.ACCESS)
    throw new BadRequestException('KT, Asset
    and Access clearances are all required');
    const lwd = x.data.lwd || ymd(), tpl =
    (t: string) => `
```

```

    }
}

/* ===== §13 HR HANDBOOK
(Feature: handbook) =====
*/
@Ctl('handbook') @Feat('handbook') class
HandbookCtl {
    constructor(private db: Db, private n:
Notifier) {}
    @Post('policies') @Roles('HR', 'HRBP',
'HR_MANAGER', 'ADMIN') async add(@Me() u:
User, @Body() b: { title: string; body:
string; version?: string; publish?: boolean
}) {
        const p = await
this.db.addRec('POLICY', { companyId:
u.companyId, ownerId: u.id, status:
b.publish === false ? 'DRAFT' :
'PUBLISHED', data: { title: b.title, body:
b.body, version: b.version || '1.0' } });
        if (p.status === 'PUBLISHED') await
this.n.send(await this.db.ids(u.companyId,
['EMPLOYEE', 'RM', 'DEPT_HEAD',
'RECRUITER', 'PANEL', 'FINANCE', ...HRS]),
{ title: `New policy: ${b.title}`, body:
'Please read and sign to acknowledge',
type: 'POLICY', link:
`/handbook/policies/${p.id}`, sms: true });
        return p;
    }
    @Post('policies/:id/publish')
@Roles('HR', 'HRBP', 'HR_MANAGER') async
pub(@Me() u: User, @Param('id') id: string)

```

```

{ const p = await this.db.rec('POLICY',
id); p.status = 'PUBLISHED'; await
this.db.recs.save(p); await
this.n.send(await this.db.ids(u.companyId,
['EMPLOYEE', 'RM', 'DEPT_HEAD',
'RECRUITER', 'FINANCE', ...HRS]), { title:
`New policy: ${p.data.title}`, type:
'POLICY', sms: true }); return p; }

@Get('policies') async list(@Me() u:
User) { const ps = await
this.db.listRec('POLICY', { companyId:
u.companyId }), acks = await
this.db.listRec('ACK', { ownerId: u.id });
return ps.filter(p => p.status ===
'PUBLISHED' || HRS.includes(u.role)).map(p
=> ({ id: p.id, status: p.status,
...p.data, acknowledged: acks.some(a =>
a.refId === p.id) })); }

@Post('policies/:id/ack') async ack(@Me()
u: User, @Param('id') id: string, @Body()
b: { signature: string }) {
    const p = await this.db.rec('POLICY',
id); if (!b.signature) throw new
BadRequestException('Digital signature
required'); if ((await
this.db.listRec('ACK', { ownerId: u.id,
refId: id })).length) return { already:
true };

    return this.db.addRec('ACK', {
companyId: u.companyId, ownerId: u.id,
refId: id, data: { signature: b.signature,
version: p.data.version, at: now(), name:
u.name, empId: u.empId } });
}

```

```

    @Get('policies/:id/status') @Roles('HR',
    'HRBP', 'HR_MANAGER', 'ADMIN') async
    status(@Me() u: User, @Param('id') id:
    string) { const acks = await
    this.db.listRec('ACK', { refId: id }), emps
    = await this.db.employees(u.companyId);
    return { acknowledged: acks.length, total:
    emps.length, pending: emps.filter(e =>
    !acks.some(a => a.ownerId === e.id)).map(e
    => ({ id: e.id, name: e.name, empId:
    e.empId }))) }; }
}

/* ===== $14 REPORTS &
ANALYTICS (Feature: reports)
===== */
@Ctl('reports') @Feat('reports')
@Roles('HR', 'HRBP', 'HR_MANAGER', 'ADMIN',
'FINANCE', 'RM') class ReportsCtl {
    constructor(private db: Db) {}
    @Get('/:name') async report(@Me() u: User,
    @Param('name') name: string,
    @Query('month') m = ym(), @Query('format')
    f: string, @Res() res: any) {
        const cid = u.companyId; let rows:
    any[] = [];
        const emps = await this.db.users.find({
    where: { companyId: cid, kind: 'EMPLOYEE' }
    });
        if (name === 'headcount') { const by:
    any = {}; emps.filter(e =>
    e.active).forEach(e => { const k =
    e.department || 'Unassigned'; by[k] =
    (by[k] || 0) + 1; }); rows =

```

```

Object.entries(by).map(([department,
headcount]) => ({ department, headcount
})); }

    else if (name === 'attendance') { const
att = await this.db.att.find({ where: {
companyId: cid, date: Like(m + '%') } });
rows = emps.map(e => { const a =
att.filter(x => x.userId === e.id), c = (s:
string) => a.filter(x => x.status ===
s).length; return { empId: e.empId, name:
e.name, present: c('PRESENT'), late:
c('LATE'), wfh: c('WFH'), outOfGeofence:
c('OUT_OF_GEOFENCE'), leave: c('LEAVE'),
absent: c('ABSENT') + c('LOP') }; }); }

    else if (name === 'attrition') { const
ex = await this.db.listRec('EXIT', {
companyId: cid }); rows = [{ exits:
ex.length, closed: ex.filter(x => x.status
=== 'CLOSED').length, activeHeadcount:
emps.filter(e => e.active).length,
attritionPct: emps.length ?
round2((ex.length / emps.length) * 100) : 0
}]; }

    else if (name === 'hiring-funnel') {
const apps = await this.db.apps.find({
where: { companyId: cid } }), by: any = {};
apps.forEach(a => (by[a.status] =
(by[a.status] || 0) + 1)); rows =
Object.entries(by).map(([stage, count]) =>
({ stage, count })); }

    else if (name === 'payroll-cost') {
rows = (await this.db.listRec('PAYRUN', {
companyId: cid })).map(r => ({ month:
r.key, status: r.status, ...r.data.totals

```

```

    )); }
    else if (name === 'mood') rows = [await
moodReport(this.db, cid, ymd())];
    else if (name === 'leave-utilisation')
{ const ap = await this.db.approvals.find({
where: { companyId: cid, type: 'LEAVE',
status: 'APPROVED' } }), by: any = {};
ap.forEach(a => (by[a.payload.code] =
(by[a.payload.code] || 0) +
a.payload.days)); rows =
Object.entries(by).map(([code, days]) => ({
code, days })); }
    else if (name === 'claims') { rows =
(await this.db.approvals.find({ where: {
companyId: cid, type: 'CLAIM' } })).map(a
=> ({ id: a.id, title: a.title, status:
a.status, total: a.payload.total, paid:
a.payload.paid, overLimit:
a.payload.overLimit })); }
    else if (name === 'kpi') { rows =
(await this.db.listRec('KPI', { companyId:
cid, key: m })).map(k => ({ employeeId:
k.ownerId, month: k.key, status: k.status,
scorePct: k.data.scorePct })); }
    else if (name === 'training') { rows =
(await this.db.listRec('TRAINING', {
companyId: cid })).map(t => ({ employeeId:
t.ownerId, title: t.data.title, provider:
t.data.provider, status: t.status })); }
    else if (name === 'policy-acks') { rows
= (await this.db.listRec('POLICY', {
companyId: cid })).map(p => ({ policy:
p.data.title, status: p.status })); }
    else throw new

```

```

BadRequestException('Reports: headcount,
attendance, attrition, hiring-funnel,
payroll-cost, mood, leave-utilisation,
claims, kpi, training, policy-acks');
    if (f === 'csv') { const h = rows[0] ?
Object.keys(rows[0]) : [];
res.setHeader('Content-Type', 'text/csv');
res.setHeader('Content-Disposition',
`attachment; filename="${name}_${m}.csv"`);
return res.send(csv([h, ...rows.map(r =>
h.map(k => r[k]))])); }
    res.json({ report: name, month: m, rows
});
  }
}

/* ===== $15 SETTINGS,
PLANS, APPROVALS INBOX, NOTIFICATIONS,
FILES ===== */
@Ctl('settings') class SettingsCtl {
  constructor(private db: Db, private n:
Notifier) {}
  @Get('plans') @Public() plans() { return
{ modules: MODULE_KEYS.concat('candidate-
portal(free)'), ats: { FREE: {
openJobPosts: 2, candidatePool: false },
PREMIUM: { openJobPosts: 5, candidatePool:
true } }, note: 'Buyers pick any
combination of modules, e.g. payroll only,
ATS only.' }; }
  @Get() async get(@Me() u: User) { const c
= await this.db.company(u.companyId);
return { company: { id: c.id, name: c.name,
state: c.state, country: c.country,

```

```

modules: c.modules, atsPlan: c.atsPlan,
deviceKey: ADMIN.includes(u.role) ?
c.deviceKey : undefined }, settings: cfg(c)
}; }

@Put() @Roles('ADMIN', 'HR_MANAGER')
async put(@Me() u: User, @Body() b: any) {
  const c = await
this.db.company(u.companyId); if (b.name)
c.name = b.name; if (b.state) c.state =
b.state; if (b.country) c.country =
b.country;
  c.settings = { ...cfg(c), ...
(b.settings || {}) }; await
this.db.companies.save(c); return
this.get(u);
}

/** Buy / drop modules and change ATS
plan (wire your payment gateway to call
this after a successful payment). */
@Post('subscribe') @Roles('SUPER_ADMIN')
async sub(@Me() u: User, @Body() b: {
modules?: string[]; atsPlan?: 'FREE' |
'PREMIUM' }) {
  const c = await
this.db.company(u.companyId); if
(b.modules) c.modules = b.modules.filter(m
=> MODULE_KEYS.includes(m)); if (b.atsPlan)
c.atsPlan = b.atsPlan; await
this.db.companies.save(c); return {
modules: c.modules, atsPlan: c.atsPlan };
}

@Post('users') @Roles('ADMIN',
'HR_MANAGER', 'HR', 'HRBP') async
addUser(@Me() u: User, @Body() b: { email:

```

```

string; name: string; role: string; rmId?:
string; hrbpId?: string; designation?:
string; department?: string; password?:
string; empId?: string; dob?: string;
phone?: string }) {
    if (!ROLES.includes(b.role) ||
['SUPER_ADMIN',
'CANDIDATE'].includes(b.role)) throw new
BadRequestException('Invalid role');
    if (await this.db.users.findOneBy({
email: b.email.toLowerCase() })) throw new
ConflictException('Email exists'); const pw
= b.password || rid(10) + '#1';
    const x = await
this.db.users.save(this.db.users.create({
...b, email: b.email.toLowerCase(),
passwordHash: await bcrypt.hash(pw, 10),
companyId: u.companyId, kind: b.role ===
'EMPLOYEE' ? 'EMPLOYEE' : 'STAFF', zodiac:
zodiac(b.dob), verified: true,
mustChangePwd: true, onboarding: 'DONE',
profile: {}, docs: [] }));
    await this.n.send(x.id, { title: 'Your
Nerkanal account is ready', body: `Login:
${x.email} / ${pw}`, type: 'AUTH' });
    return { user: clean(x), tempPassword: pw
};
}

@Get('users') @Roles('ADMIN',
'HR_MANAGER', 'HR', 'HRBP', 'RM') async
users(@Me() u: User) { return (await
this.db.users.find({ where: { companyId:
u.companyId }, order: { name: 'ASC' }
})).map(clean); }

```

```

    @Put('users/:id') @Roles('ADMIN',
    'HR_MANAGER', 'HR', 'HRBP') async
    upUser(@Me() u: User, @Param('id') id:
    string, @Body() b: any) { const x = await
    this.db.users.findOneByOrFail({ id,
    companyId: u.companyId }); for (const k of
    ['role', 'rmId', 'hrbpId', 'designation',
    'department', 'active']) if (b[k] !==
    undefined) (x as any)[k] = b[k]; return
    clean(await this.db.users.save(x)); }
    }

    @Ctl('approvals') class ApprovalsCtl {
        constructor(private db: Db, private ap:
        Approvals) {}

        /** Everything waiting on ME (RM → HR →
        Admin → Finance chain). */
        @Get('inbox') async inbox(@Me() u: User)
        { const pend = await
        this.db.approvals.find({ where: {
        companyId: u.companyId, status: 'PENDING'
        }, order: { createdAt: 'DESC' } }); const
        out = []; for (const a of pend) if (await
        this.ap.canAct(a, u)) out.push(await
        this.ap.view(a)); return out; }

        @Get('mine') async mine(@Me() u: User) {
        return Promise.all((await
        this.db.approvals.find({ where: {
        requesterId: u.id }, order: { createdAt:
        'DESC' } })).map(a => this.ap.view(a))); }

        @Get('/:id') async one(@Me() u: User,
        @Param('id') id: string) { const a = await
        this.db.approvals.findOneBy({ id }); if (!a
        || (a.companyId !== u.companyId && u.role
        !== 'SUPER_ADMIN')) throw new

```

```

NotFoundException(); return
this.ap.view(a); }

@Post('/:id/act') async act(@Me() u: User,
@Param('id') id: string, @Body() b: {
approve: boolean; remarks?: string }) {
return this.ap.act(id, u, !!b.approve,
b.remarks); }
}

@Ctl('notifications') class
NotificationsCtl {
constructor(private db: Db) {}
@Get() list(@Me() u: User,
@Query('unread') unread?: string) { return
this.db.notes.find({ where: { userId: u.id,
...(unread ? { read: false } : {}) },
order: { createdAt: 'DESC' }, take: 100 });
}

@Post('read-all') async readAll(@Me() u:
User) { await this.db.notes.update({
userId: u.id }, { read: true }); return {
ok: true }; }

@Post('/:id/read') async read(@Me() u:
User, @Param('id') id: string) { await
this.db.notes.update({ id, userId: u.id },
{ read: true }); return { ok: true }; }
}

@Ctl('common') class CommonCtl {
constructor(private db: Db) {}
@Post('files') @UseInterceptors(upload)
file(@UploadedFile() f: any) { if (!f)
throw new BadRequestException('multipart
field "file" required'); return { url:
`/files/${f.filename}`, name:
f.originalname, size: f.size }; }
}

```

```

    @Get('calendar') async cal(@Me() u: User)
    { return (await this.db.meetings.find({
    where: { companyId: u.companyId }, order: {
    startsAt: 'ASC' } })).filter(m =>
    m.attendeeIds.includes(u.id) && m.status
    !== 'DONE').map(m => ({ id: m.id, title:
    m.title, kind: m.kind, startsAt:
    m.startsAt, durationMin: m.durationMin,
    link: m.link })); }

    @Get('team') async team(@Me() u: User) {
    return (await this.db.users.find({ where: {
    companyId: u.companyId, active: true }
    })).map(x => ({ id: x.id, name: x.name,
    role: x.role, empId: x.empId, department:
    x.department })); }
}

/* ===== JARVIS – the AI
assistant that acts inside Nerkanal
===== */

type Tool = { name: string; description:
string; props: Record<string, any>;
required?: string[]; roles?: string[];
feature?: string };

const T = (name: string, description:
string, props: any = {}, required: string[]
= [], roles?: string[], feature?: string):
Tool => ({ name, description, props,
required, roles, feature });

const str = (d = '') => ({ type: 'string',
description: d });

@Injectable() class Jarvis {
    private tools: Tool[] = [
        T('leave_balance', 'Get my leave

```

```
balances', {}, [], undefined, 'leave'),
  T('apply_leave', 'Apply for leave.
Dates are YYYY-MM-DD. Code like CL, SL,
PL.', { code: str(), from: str(), to:
str(), reason: str(), proofUrl: str() },
['code', 'from', 'reason'], undefined,
'leave'),
  T('punch', 'Mark my attendance (IN or
OUT)', { type: { type: 'string', enum:
['IN', 'OUT'] }, mood: { type: 'string',
enum: Object.keys(MOODS) } }, ['type'],
undefined, 'attendance'),
  T('pending_approvals', 'List approvals
waiting for me'),
  T('decide_approval', 'Approve or reject
an approval by id. Confirm with the user
before using.', { id: str(), approve: {
type: 'boolean' }, remarks: str() }, ['id',
'approve']),
  T('search_jobs', 'Search jobs
(candidates only)', { q: str('title or
skill') }, ['q'], ['CANDIDATE']),
  T('apply_job', 'Apply to a job by id
(candidates only)', { jobId: str() },
['jobId'], ['CANDIDATE']),
  T('start_mock_interview', 'Start a mock
interview', { role: str() }, [],
['CANDIDATE']),
  T('create_requisition', 'Raise a
manpower requisition', { title: str(),
headcount: { type: 'number' },
justification: str() }, ['title',
'headcount'], ['DEPT_HEAD', 'RM',
'HR_MANAGER'], 'ats'),
```

```

        T('payslip', 'Get my payslip for a
month YYYY-MM', { month: str() }, [],
undefined, 'payroll'),
        T('raise_complaint', 'Raise an
anonymous complaint', { subject: str(),
body: str() }, ['subject', 'body'],
undefined, 'complaints'),
        T('team_mood', 'Daily mood report for
my team (RM/HR)', { date: str() }, [],
['RM', ...HRS], 'attendance'),
        T('schedule_meeting', 'Schedule a
meeting with colleagues by email', { title:
str(), startsAt: str('ISO datetime'),
emails: { type: 'array', items: { type:
'string' } } }, ['title', 'startsAt',
'emails']),
        T('my_notifications', 'Show my unread
notifications'),
        T('horoscope', "Today's horoscope for
me"),
    ];
    constructor(private mr: ModuleRef,
private db: Db, private ap: Approvals) {}
    private c<T>(k: new (...a: any[]) => T):
T { return this.mr.get(k, { strict: false
}); }
    private async exec(name: string, a: any,
u: User): Promise<any> {
        const t = this.tools.find(x => x.name
=== name); if (!t) throw new
BadRequestException('Unknown tool');
        if (t.roles &&
!t.roles.includes(u.role) && u.role !==
'SUPER_ADMIN') throw new

```

```
ForbiddenException(`Your role (${u.role})
cannot do that`);
    if (t.feature && u.companyId && !(await
this.db.company(u.companyId)).modules.inclu
des(t.feature)) throw new
ForbiddenException(`Module ${t.feature} is
not purchased`);
    switch (name) {
        case 'leave_balance': return
this.c(LeaveCtl).balances(u);
        case 'apply_leave': return
this.c(LeaveCtl).apply(u, a);
        case 'punch': return
this.c(AttendanceCtl).punch(u, { type:
a.type, mode: 'APP', mood: a.mood });
        case 'pending_approvals': return
this.c(ApprovalsCtl).inbox(u);
        case 'decide_approval': return
this.ap.act(a.id, u, !!a.approve,
a.remarks);
        case 'search_jobs': return (await
this.c(JobsCtl).search(u, a.q,
'')).results.slice(0, 8).map((j: Job) => ({
id: j.id, title: j.title, company:
j.companyName, source: j.source, applyUrl:
j.applyUrl }));
        case 'apply_job': return
this.c(JobsCtl).apply(u, a.jobId);
        case 'start_mock_interview': return
this.c(CandidateCtl).mockStart(u, a);
        case 'create_requisition': return
this.c(AtsCtl).req(u, a);
        case 'payslip': return
this.c(PayrollCtl).payslip(u, a.month ||
```

```

ym());
    case 'raise_complaint': return
this.c(ComplaintsCtl).raise(u, { ...a,
anonymous: true });
    case 'team_mood': return
moodReport(this.db, u.companyId, a.date ||
ymd(), HRS.includes(u.role) ? undefined :
u.id);
    case 'schedule_meeting': { const us =
await this.db.users.find({ where: { email:
In((a.emails || []).map((e: string) =>
e.toLowerCase()))), companyId: u.companyId }
}); const m = await
this.ap.meets(u.companyId, u.id, 'MEETING',
a.title, a.startsAt, 30, us.map(x =>
x.id)); return { link: m.link, invited:
us.map(x => x.name) }; }
    case 'my_notifications': return
(await this.db.notes.find({ where: {
userId: u.id, read: false }, order: {
createdAt: 'DESC' }, take: 10 })).map(n =>
({ title: n.title, body: n.body }));
    case 'horoscope': return u.zodiac ?
horoscope(u.zodiac) : { error: 'No zodiac
on profile' };
  }
}
  async chat(u: User, message: string,
history: any[] = []) {
    const actions: any[] = [];
    if (!ENV('ANTHROPIC_API_KEY')) return {
reply: await this.fallback(u, message,
actions), actions, mode: 'rules (set
ANTHROPIC_API_KEY for full Jarvis)' };

```

```

    const sys = `You are Jarvis, the
assistant inside Nerkanal (HRMS/ATS). You
act on behalf of ${u.name} (role
${u.role}). Today is ${ymd()}. Use tools to
take actions. Never invent ids or data –
fetch them with tools first. Ask for
confirmation before approving/rejecting,
raising complaints, or any irreversible
step. Be brief.`;
    const msgs: any[] =
[...history.slice(-10), { role: 'user',
content: message }], tools =
this.tools.filter(t => !t.roles ||
t.roles.includes(u.role) || u.role ===
'SUPER_ADMIN').map(t => ({ name: t.name,
description: t.description, input_schema: {
type: 'object', properties: t.props,
required: t.required } }));
    for (let i = 0; i < 6; i++) {
        const r: any = await
fetch('https://api.anthropic.com/v1/message
s', { method: 'POST', headers: { 'x-api-
key': ENV('ANTHROPIC_API_KEY'), 'anthropic-
version': '2023-06-01', 'content-type':
'application/json' }, body:
JSON.stringify({ model: ENV('JARVIS_MODEL',
'claude-sonnet-5-5'), max_tokens: 1024,
system: sys, tools, messages: msgs })
}).then(x => x.json());
        if (r.error) throw new
BadRequestException('Jarvis LLM error: ' +
r.error.message);
        const uses = (r.content ||
[]).filter((b: any) => b.type ===

```

```

'tool_use');
    if (r.stop_reason !== 'tool_use' ||
!uses.length) return { reply: (r.content ||
[]).filter((b: any) => b.type ===
'text').map((b: any) => b.text).join('\n'),
actions, history: [...msgs, { role:
'assistant', content: r.content }] };
    msgs.push({ role: 'assistant',
content: r.content }); const results: any[]
= [];
    for (const b of uses) { let out: any;
try { out = await this.exec(b.name, b.input
|| {}, u); } catch (e) { out = { error:
e.response?.message || e.message }; }
actions.push({ tool: b.name, input:
b.input, ok: !out?.error }); results.push({
type: 'tool_result', tool_use_id: b.id,
content: JSON.stringify(out).slice(0, 6000)
}); }
    msgs.push({ role: 'user', content:
results });
    }
    return { reply: 'I could not finish
that – please try again.', actions };
    }
    private async fallback(u: User, m:
string, actions: any[]) {
        const s = m.toLowerCase(), run = async
(n: string, a: any = {}) => {
actions.push({ tool: n }); try { return
await this.exec(n, a, u); } catch (e) {
return { error: e.response?.message ||
e.message }; } };
        if (/balance/.test(s)) return

```

```

JSON.stringify(await run('leave_balance'));
    if (/pending|approval/.test(s)) return
JSON.stringify((await
run('pending_approvals')) || []);
    if (/punch in|check in|mark
attendance/.test(s)) return
JSON.stringify(await run('punch', { type:
'IN' }));
    if (/punch out|check out/.test(s))
return JSON.stringify(await run('punch', {
type: 'OUT' }));
    if (/horoscope/.test(s)) return
JSON.stringify(await run('horoscope'));
    if (/notification/.test(s)) return
JSON.stringify(await
run('my_notifications'));
    if (/payslip|salary/.test(s)) return
JSON.stringify(await run('payslip'));
    if (/mood/.test(s)) return
JSON.stringify(await run('team_mood'));
    return 'I can do: leave balance,
pending approvals, punch in/out, horoscope,
notifications, payslip, team mood. Add
ANTHROPIC_API_KEY to unlock full natural-
language actions (apply leave, search/apply
jobs, raise requisition, schedule
meetings...).';
}
}
@Ctl('jarvis') class JarvisCtl {
    constructor(private j: Jarvis) {}
    @Post('chat') chat(@Me() u: User, @Body()
b: { message: string; history?: any[] }) {
    if (!b.message) throw new

```

```

BadRequestException('message required');
return this.j.chat(u, b.message, b.history
|| []); }
}

/* ===== Built-in
"Nerkanal Meet" page (WebRTC mesh, Google-
Meet-like) ===== */
@Controller() class MeetCtl {
    @Public() @Get('meet/:roomId')
page(@Param('roomId') roomId: string,
@Res() res: any) {
    res.type('html').send(`<!doctype html>
<meta name=viewport content="width=device-
width,initial-scale=1"><title>Nerkanal
Meet</title>
<style>body{margin:0;background:#0b1f4b;col
or:#fff;font-family:system-
ui}#g{display:grid;grid-template-
columns:repeat(auto-
fit,minmax(260px,1fr));gap:8px;padding:8px}
video{width:100%;background:#000;border-
radius:10px}header{padding:10px
16px;background:#06143a;color:#e1a82d;font-
weight:700}#bar{position:fixed;bottom:0;left:0;right:0;padding:10px;text-
align:center;background:#06143a}button{back
ground:#e1a82d;border:0;border-
radius:8px;padding:8px 14px;margin:0
4px;font-weight:700}</style>
<header>Nerkanal Meet · room
${roomId.replace(/^[^w-]/g, '')}</header>
<div id=g></div><div id=bar><button
id=m>Mute</button><button

```

```

id=c>Camera</button><button
onclick="location='/'">Leave</button></div>
<script src="/socket.io/socket.io.js">
</script><script>
const room="${roomId.replace(/^[^w-]/g,
'')}" ,q=new
URLSearchParams(location.search);let
token=q.get('token')||localStorage.getItem(
'nk_token')||prompt('Paste your Nerkana1
access
token');localStorage.setItem('nk_token',tok
en);
const name=prompt('Your
name')||'Guest',s=io({auth:{token}}),peers=
{},g=document.getElementById('g');let me;
const cfg={iceServers:
[{urls:'stun:stun.l.google.com:19302'}]};
function tile(id,stream){let
v=document.getElementById(id);if(!v)
{v=document.createElement('video');v.id=id;
v.autoplay=true;v.playsInline=true;g.append
Child(v)}v.srcObject=stream;return v}
function pc(id){const p=new
RTCPeerConnection(cfg);peers[id]=p;me.getTr
acks().forEach(t=>p.addTrack(t,me));p.onice
candidate=e=>e.candidate&&s.emit('signal',
{to:id,data:
{ice:e.candidate}});p.ontrack=e=>tile(id,e.
streams[0]);return p}
(async()=>{me=await
navigator.mediaDevices.getUserMedia({video:
true,audio:true});tile('me',me).muted=true;
s.emit('join',{roomId:room,name});
s.on('denied',m=>{alert(m);location='/'});

```

```

s.on('peers', async l=>{for(const x of l)
{const p=pc(x.id);const o=await
p.createOffer();await
p.setLocalDescription(o);s.emit('signal',
{to:x.id,data:
{sdp:p.localDescription}})}));
s.on('signal', async({from,data})=>{const
p=peers[from]||pc(from);if(data.sdp){await
p.setRemoteDescription(data.sdp);if(data.sd
p.type==='offer'){const a=await
p.createAnswer();await
p.setLocalDescription(a);s.emit('signal',
{to:from,data:
{sdp:p.localDescription}})}}if(data.ice)try
{await p.addIceCandidate(data.ice)}catch(e)
{}});
s.on('peer-left',({id})=>
{peers[id]?.close();delete
peers[id];document.getElementById(id)?.remo
ve()});
m.onclick=
()=>me.getAudioTracks().forEach(t=>t.enable
d=!t.enabled);c.onclick=
()=>me.getVideoTracks().forEach(t=>t.enable
d=!t.enabled)}());
</script>`);
}
@Public() @Get() home(@Res() res: any) {
  const logo =
fs.existsSync('public/logo.png') ? '' : '';
  res.type('html').send(`<body
style="font-family:system-ui;text-
align:center;padding:48px;background:#fff;c

```

```

olor:#0B1F4B">${logo}<h1>Nerkanal</h1>
<p>HRMS · ATS · Candidate Portal API is
running.</p><p><a href="/docs"
style="color:#b8860b">Open API console
(Swagger)</a></p></body>`);
    }
}

```

```

/* ===== CRON – every
reminder, escalation and auto-trigger in
the spec ===== */
@Injectable() class Cron_ {
    constructor(private db: Db, private n:
Notifier, private ap: Approvals, private
ext: ExternalJobs) {}
    private async companies() { return
this.db.companies.find(); }
    /** 08:00 – probation ending in 2 days →
HR (confirm or start separation) */
    @Cron('0 8 * * *') async probation() {
for (const c of await this.companies()) for
(const e of await this.db.employees(c.id))
if (e.probationState === 'ON' &&
e.probationEnd && diffDays(ymd(),
e.probationEnd) === 2) await
this.n.send(await this.db.ids(c.id, HRS), {
title: `Probation ends in 2 days: ${e.name}
${e.empId}`, body: 'Confirm the employee
or initiate separation', type: 'PROBATION',
dedupe: `prob:${e.id}:${e.probationEnd}`,
link: '/onboarding/probation', sms: true
}); }
    /** 00:30 – mark yesterday's absentees; 3
days uninformed → day-4 show-cause; no

```

```

reply in 7 days → termination initiated */
@Cron('30 0 * * *') async absence() {
  const y = addDays(ymd(), -1);
  for (const c of await this.companies())
  {
    if (!(c.modules ||
  []).includes('attendance')) continue; const
  s = cfg(c);
    for (const e of await
  this.db.employees(c.id)) {
      if (e.onboarding === 'PENDING' ||
  (e.doj && e.doj > y)) continue; let streak
  = 0;
      for (let d = y, i = 0; i < 10; d =
  addDays(d, -1), i++) {
        if
  (s.weekOff.includes(P(d).getDay()))
  continue; const r = await
  this.db.att.findOneBy({ userId: e.id, date:
  d });
        if (r && ['LEAVE',
  'LOP'].includes(r.status)) break; if
  (r?.inAt) break;
        if (d === y && !r) await
  this.ap.markAtt(e.id, c.id, d, { status:
  'ABSENT', mode: 'SYSTEM' }); streak++;
      }
      if (streak === 3) { // the 4th day
  starts today
        const sc = await
  this.db.addRec('SHOWCAUSE', { companyId:
  c.id, ownerId: e.id, status: 'OPEN', data:
  { issuedAt: now(), dueBy: addDays(ymd(),
  7), reason: '3 consecutive days of

```

```

uninformed absence' } }));
      await this.n.send([e.id, e.rmId,
...(await this.db.ids(c.id,
HRS))].filter(Boolean), { title: `Show-
cause notice: ${e.name}`, body: '3 days
uninformed absence. Reply within 7 days.',
type: 'DISCIPLINE', dedupe: `sc:${sc.id}`,
link: `/attendance/showcause`, sms: true
});
    }
  }
  for (const sc of await
this.db.listRec('SHOWCAUSE', { companyId:
c.id, status: 'OPEN' })) if
(diffDays(sc.createdAt, ymd()) >= 7) {
    sc.status = 'ESCALATED'; await
this.db.recs.save(sc); const e = await
this.db.user(sc.ownerId);
    await this.ap.create('TERMINATION',
e, { reason: 'No response to show-cause
notice (uninformed absence) within 7 days'
}, ['HR_MANAGER'], `Termination initiated
(policy): ${e.name}`);
  }
}
}
/** 11:30 – daily mood report to every RM
and to HR */
@Cron('30 11 * * *') async mood() { for
(const c of await this.companies()) { if
(!c.modules?.includes('attendance'))
continue; const d = ymd(); for (const rm of
await this.db.byRole(c.id, ['RM'])) { const
r = await moodReport(this.db, c.id, d,

```

```

rm.id); if (r.responded) await
this.n.send(rm.id, { title: `Team mood
today: ${r.enthusiasmScore}/5`, body:
`${r.responded}/${r.teamSize} responded.
${r.concerns.length ? 'Check in with: ' +
r.concerns.join(', ') : ''}`, type: 'MOOD',
dedupe: `mood:${rm.id}:${d}`, email: false
}); } const all = await moodReport(this.db,
c.id, d); if (all.responded) await
this.n.send(await this.db.ids(c.id, HRS), {
title: `Company mood today:
${all.enthusiasmScore}/5`, body:
JSON.stringify(all.counts), type: 'MOOD',
dedupe: `mood:hr:${c.id}:${d}`, email:
false }); } }

/** every minute – live-tracking watchdog
(location off → RM, still off after 30 min
→ HR) + water snoozes */
@Cron('* * * * *') async minute() {
  for (const t of await
this.db.recs.find({ where: { type: 'TRIP',
status: 'ACTIVE' } }))) {
    const d = t.data, c = await
this.db.companies.findOneBy({ id:
t.companyId }), s = cfg(c), u = await
this.db.users.findOneBy({ id: t.ownerId });
    if (!u) continue;
    const silent = Date.now() - +new
Date(d.lastPingAt) > 3 * 6e4; if ((silent
|| d.enabled === false) && !d.offSince)
d.offSince = now();
    if (d.offSince) {
      if (!d.rmNotified) { d.rmNotified =
true; await this.n.send(u.rmId || (await

```

```

this.db.ids(c.id, HRS)), { title:
`${u.name}: location off / no signal`,
type: 'TRACKING', sms: true }); }
    if (!d.hrNotified && (Date.now() -
+new Date(d.offSince)) / 6e4 >=
s.locationOffEscalateMin) { d.hrNotified =
true; await this.n.send(await
this.db.ids(c.id, HRS), { title:
`ESCALATION: ${u.name} location off for
${s.locationOffEscalateMin}+ min`, type:
'TRACKING', sms: true }); }
    t.data = { ...d }; await
this.db.recs.save(t);
  }
}
  for (const sn of await
this.db.recs.find({ where: { type:
'SNOOZE', status: 'OPEN' } })) if (new
Date(sn.data.dueAt) <= new Date()) {
sn.status = 'DONE'; await
this.db.recs.save(sn); await
this.n.send(sn.ownerId, { title: '💧 Time
for water', body: 'Tap “Drank” or “Later”',
type: 'WATER', email: false }); }
}
  /** every 2h in working hours – hydration
nudge for employees who are in office today
*/
  @Cron('0 10-18/2 * * 1-6') async water()
{ const d = ymd(), h = new
Date().getHours(); for (const c of await
this.companies()) { if
(!c.modules?.includes('engagement'))
continue; const ids = (await

```

```

this.db.att.find({ where: { companyId:
c.id, date: d } })).filter(a => a.inAt &&
!a.outAt).map(a => a.userId); await
this.n.send(ids, { title: '💧 Hydration
break', body: 'Have a glass of water',
type: 'WATER', dedupe: `water:${d}:${h}`,
email: false }); } }

/** 09:00 on the 1st – RMs set KPIs;
monthly 1:1s scheduled; every 2nd month
skill-gap reminder */
@Cron('0 9 1 * *') async monthStart() {
  for (const c of await this.companies())
  {
    if
(c.modules?.includes('performance')) await
this.n.send(await this.db.ids(c.id,
['RM']), { title: `Set ${ym()} KPIs for
your team today`, body: 'Unset KPIs are
forwarded to HR tomorrow', type: 'KPI',
dedupe: `kpi1:${c.id}:${ym()}` });
    if
(c.modules?.includes('engagement')) await
scheduleOneOnOnes(this.db, this.ap, c.id);
    if (c.modules?.includes('training')
&& new Date().getMonth() % 2 === 0) await
this.n.send(await this.db.ids(c.id,
['RM']), { title: 'Bi-monthly skill-gap
review due', body: 'Analyse your team and
share skill gaps', type: 'TRAINING',
dedupe: `sg:${c.id}:${ym()}` });
    if (c.modules?.includes('leave') &&
new Date().getMonth() === 0) await
creditYear(this.db, c.id);
  }
}

```

```

    }
    /** 09:00 on the 2nd – KPIs still not set
are forwarded to HR */
    @Cron('0 9 2 * *') async kpiEscalate() {
for (const c of await this.companies()) {
if (!c.modules?.includes('performance'))
continue; for (const e of (await
this.db.employees(c.id)).filter(x =>
x.rmId)) if (!(await this.db.listRec('KPI',
{ ownerId: e.id, key: ym() }))).length)
await this.n.send(await this.db.ids(c.id,
HRS), { title: `KPI not set for ${e.name}
(${ym()})`, body: 'Reporting manager missed
the 1st-day deadline', type: 'KPI', dedupe:
`kpi2:${e.id}:${ym()}` }); } }
    /** 09:00 daily – PIP ending in 2 days;
POSH/training events in 7 days → HR */
    @Cron('0 9 * * *') async daily() {
for (const c of await this.companies())
{
    if
(c.modules?.includes('performance')) for
(const p of await this.db.listRec('PIP', {
companyId: c.id, status: 'ACTIVE' })) if
(diffDays(ymd(), p.data.endDate) === 2) {
const e = await this.db.user(p.ownerId);
await this.n.send([...(await
this.db.ids(c.id, HRS)),
e.rmId].filter(Boolean), { title: `PIP ends
in 2 days: ${e.name}`, body: 'Record
outcome: success / extend / separate',
type: 'PIP', dedupe:
`pip2:${p.id}:${p.data.endDate}`, link:
`/performance/pip/${p.id}`, sms: true }); }

```

```

        if (c.modules?.includes('training'))
        for (const t of await
        this.db.listRec('TRAINING_CAL', {
        companyId: c.id })) if (diffDays(ymd(),
        t.key) === 7) await this.n.send(await
        this.db.ids(c.id, HRS), { title: `Training
        in 7 days: ${t.data.title}`, body: t.key,
        type: 'TRAINING', dedupe: `tcal:${t.id}`
        });
    }
}

/** 06:00 – refresh external jobs for
configured queries
(JOB_SYNC_QUERIES="developer,accountant,sal
es") */
@Cron('0 6 * * *') async jobs() { for
(const q of ENV('JOB_SYNC_QUERIES',
'developer,marketing,sales,hr').split(','))
await this.ext.search(q.trim()); }
}

/* ===== SEED DATA (first
run) ===== */
@Injectable() class Seeder implements
OnModuleInit {
    constructor(private db: Db) {}
    async onModuleInit() {
        if (!(await
        this.db.listRec('LAW')).length) for (const
        l of SEED_LAWS) await this.db.addRec('LAW',
        { data: l });
        if (await this.db.companies.count())
        return;
        const c = await

```

```

this.db.companies.save(this.db.companies.create({ name: 'Nerkanal Demo Pvt Ltd',
state: 'Tamil Nadu', country: 'IN',
modules: MODULE_KEYS, atsPlan: 'PREMIUM',
deviceKey: 'demo-device-key', settings: {
geofences: [{ name: 'HQ', lat: 13.0827,
lng: 80.2707, radiusM: 200 } ] } }));
    const mk = async (email: string, name:
string, role: string, extra: any = {}) =>
this.db.users.save(this.db.users.create({
email, name, role, kind: role ===
'EMPLOYEE' || role === 'RM' ? 'EMPLOYEE' :
'STAFF', companyId: c.id, passwordHash:
await bcrypt.hash('Nerkanal@123', 10),
verified: true, onboarding: 'DONE',
profile: {}, docs: [], dob: '1994-08-15',
zodiac: zodiac('1994-08-15'), ...extra }));
    const sa = await
mk('superadmin@nerkanal.app', 'Super
Admin', 'SUPER_ADMIN'), ad = await
mk('admin@nerkanal.app', 'Admin', 'ADMIN');
    const hrm = await
mk('hrmanager@nerkanal.app', 'HR Manager',
'HR_MANAGER'), hrbp = await
mk('hrbp@nerkanal.app', 'HRBP Priya',
'HRBP'), hr = await mk('hr@nerkanal.app',
'HR Executive', 'HR');
    await mk('recruiter@nerkanal.app',
'Recruiter Ravi', 'RECRUITER'); await
mk('finance@nerkanal.app', 'Finance
Fatima', 'FINANCE'); await
mk('depthead@nerkanal.app', 'Dept Head
Dev', 'DEPT_HEAD', { department:
'Engineering' }); await

```

```

mk('panel@nerkanal.app', 'Panel Pooja',
'PANEL');
    const rm = await mk('rm@nerkanal.app',
'Manager Mohan', 'RM', { empId: 'NK0001',
designation: 'Engineering Manager',
department: 'Engineering', doj: '2024-01-
10', hrbpId: hrbp.id, probationState:
'CONFIRMED', rmId: hrm.id });
    const emp = await
mk('employee@nerkanal.app', 'Employee
Esha', 'EMPLOYEE', { empId: 'NK0002',
designation: 'Software Engineer',
department: 'Engineering', doj: '2025-06-
01', rmId: rm.id, hrbpId: hrbp.id,
probationState: 'CONFIRMED', profile: {
bankAccount: '1234567890', ifsc:
'HDFC00000001', uan: '100000000001' } });
    await
this.db.users.save(this.db.users.create({
email: 'candidate@nerkanal.app', name:
'Candidate Karthik', role: 'CANDIDATE',
kind: 'CANDIDATE', passwordHash: await
bcrypt.hash('Nerkanal@123', 10), verified:
true, dob: '1998-03-02', phone:
'9999999999', zodiac: zodiac('1998-03-02'),
profile: {}, docs: [], profilePct: 0 }));
    const lt = [['CL', 'Casual Leave', 12,
false, true], ['SL', 'Sick Leave', 8,
false, true], ['PL', 'Privilege Leave', 15,
true, true], ['CO', 'Comp-off', 0, false,
true], ['LOP', 'Loss of Pay', 0, false,
false]] as const;
    for (const [code, name, quota, carry,
paid] of lt) { await

```

```

this.db.leaveTypes.save(this.db.leaveTypes.create({
  companyId: c.id, code, name, quota, carry, paid, applicableTo: ['ALL'],
  proofAfterDays: 3 })); }
  const year = String(new Date().getFullYear());
  for (const u of [rm, emp]) for (const [code, , quota] of lt) if (quota) await
this.db.leaveBal.save(this.db.leaveBal.create({
  userId: u.id, code, year, bal: quota }));
  const st = await
this.db.addRec('STRUCT', { companyId: c.id, status: 'ACTIVE',
  data: { name: 'Standard Structure', components: SAMPLE_STRUCT } });
  await this.db.addRec('EMPSAL', { companyId: c.id, ownerId: emp.id,
  refId: st.id, data: { ctc: 60000 } });
  await this.db.addRec('EMPSAL', { companyId: c.id, ownerId: rm.id,
  refId: st.id, data: { ctc: 120000 } });
  await this.db.addRec('POLICY', { companyId: c.id, ownerId: hr.id,
  status: 'PUBLISHED', data: { title: 'Code of Conduct', version: '1.0',
  body: 'All employees shall act with integrity, respect and professionalism...' } });
  await this.db.addRec('POLICY', { companyId: c.id, ownerId: hr.id,
  status: 'PUBLISHED', data: { title: 'POSH Policy', version: '1.0',
  body: 'Nerkanal has zero tolerance for sexual harassment at the workplace...' } });
  log.log('Seeded demo company. Logins

```

```

(password Nerkanal@123): superadmin@ /
admin@ / hrmanager@ / hrbp@ / hr@ /
recruiter@ / finance@ / depthead@ / panel@
/ rm@ / employee@ / candidate@ →
nerkanal.app');
    }
}

/* ===== APP MODULE +
BOOTSTRAP ===== */
const dbOptions: any = ENV('DATABASE_URL')
? { type: 'postgres', url:
ENV('DATABASE_URL'), ssl: ENV('DB_SSL') ===
'true' ? { rejectUnauthorized: false } :
false } : { type: 'better-sqlite3',
database: ENV('SQLITE_FILE', 'nerkanal.db')
};
@Global() @Module({
  imports: [
    TypeOrmModule.forRoot({ ...dbOptions,
entities: ENTITIES, synchronize:
ENV('DB_SYNC', 'true') === 'true' }), //
use migrations in production
    TypeOrmModule.forFeature(ENTITIES),
    JwtModule.register({ global: true,
secret: ENV('JWT_SECRET', 'nerkanal-dev-
secret'), signOptions: { expiresIn: '7d' }
}),
    ScheduleModule.forRoot(),
  ],
  controllers: [AuthCtl, CandidateCtl,
JobsCtl, AtsCtl, OnboardingCtl,
AttendanceCtl, LeaveCtl, PayrollCtl,
TrackingCtl, ClaimsCtl, PerformanceCtl,

```

```

EngagementCtl, FeedCtl, TrainingCtl,
ComplaintsCtl, ExitCtl, HandbookCtl,
ReportsCtl, SettingsCtl, ApprovalsCtl,
NotificationsCtl, CommonCtl, JarvisCtl,
MeetCtl],
  providers: [Db, Notifier, Approvals,
ExternalJobs, VideoAI, Jarvis, AppGateway,
Cron_, Seeder, { provide: APP_GUARD,
useClass: AccessGuard }],
  exports: [Db],
})
class AppModule {}

async function bootstrap() {
  const app = await
NestFactory.create<NestExpressApplication>
(AppModule, { logger: ['log', 'warn',
'error'] });
  app.enableCors(); app.useGlobalPipes(new
ValidationPipe({ transform: true,
whitelist: false }));
  fs.mkdirSync('uploads', { recursive: true
}); fs.mkdirSync('public', { recursive:
true });

  app.useStaticAssets(path.resolve('uploads')
, { prefix: '/files/' });
  app.useStaticAssets(path.resolve('public'))
;
  const doc =
SwaggerModule.createDocument(app, new
DocumentBuilder().setTitle('Nerkanal
API').setDescription('HRMS + ATS +
Candidate Portal – 15 modules +

```

```
Jarvis').setVersion('1.0').addBearerAuth().
build());
  SwaggerModule.setup('docs', app, doc, {
    customSiteTitle: 'Nerkanal API',
    swaggerOptions: { persistAuthorization:
true } });
  const port = +ENV('PORT', '3000'); await
app.listen(port, '0.0.0.0');
log.log(`Nerkanal running on
http://localhost:${port} (API console:
/docs)`);
}
bootstrap();
```